PaperPass[免费版]AIGC检测报告

简明打印版

AIGC总体疑似度(高+中+轻)：37.06%
AIGC总体疑似度(高+中+轻)：25.95%（加权计算）

高度疑似AIGC占全文比：	3.68%
中度疑似AIGC占全文比：	30.28%
轻度疑似AIGC占全文比：	3.1%
不予检测文字占比：	33.91%

检测版本：免费版(仅检测中文)
报告编号：ER8669F1F902D2B81
论文题目：毕业论文-刘芷悠
论文作者：佚名
论文字数：31382
段落个数：1018
句子个数：1550
片段个数：234
提交时间：2026-4-29 20:26:42
查询真伪：https://www.paperpass.com/check

疑似度分布图：

正文中片段的不同颜色表示不同的疑似度范围

- 红色：AIGC生成疑似度在70%以上（高度疑似）
- 橙色：AIGC生成疑似度在60%~70%（中度疑似）
- 紫色：AIGC生成疑似度在50%~60%（轻度疑似）
- 黑色：AIGC生成疑似度在50%以下
- 灰色：过短片段、标题和英文等不予检测的文字

注：AIGC检测可有效识别文本是否部分或全部由AI模型生成，检测结果与论文质量无关、仅表示论文中内容片段存在AI生成可能性的概率。

加权计算公式：（片段1疑似度 + 片段2疑似度 + ... + 片段n疑似度）/ n
片段疑似度范围0.0~1.0，合格片段按照0计算，不计算不予检测文字

浙江工商大學

本科毕业论文（设计）正文

题目：基于实时状态的校园学习空间综合服务平台设计与实现

专　业：计算机科学与技术

班　级：计科 2202

学　号：2201110704

二〇二六 年 五 月

摘 要

随着智慧校园建设不断推进，高校学习空间管理从传统静态管理逐步转向动态化、精细化与服务化。针对现有校园学习空间系统中普遍存在的功能分散、信息更新不及时、预约与使用环节脱节、反馈处理闭环不完整等问题，本文设计并实现了一套基于实时状态的校园学习空间综合服务平台。系统采用前后端分离架构，前端基于 Vue 3、Pinia 与 Vue Router 构建多角色交互界面，后端基于 Spring Boot、MyBatis-Plus、MySQL 与 Redis 实现业务逻辑与数据持久化，并结合 WebSocket 支持状态实时推送。

本系统围绕校园学习空间使用全流程，构建了“查询-预约-签到-使用-反馈-治理”的业务闭环。用户端实现了教学楼与教室浏览、课程占用信息查看、学习空间预约、二维码签到核销、教室评价、设施报修、协作组队、学习计划管理、资源市场及校园导航等功能；管理端实现了报修工单管理、评价审核、资源审核、教室与课程管理、教学楼管理、预约规则配置以及扫码设备管理等功能。通过统一接口规范、角色权限控制和模块化设计，平台在功能完整性与可维护性方面具备较好工程实践价值。

本文在需求分析基础上给出了系统总体架构、功能模块划分与数据库设计思路，并结合关键业务流程对主要实现方案进行说明。系统测试结果表明，平台能够稳定支撑核心业务场景，满足学习空间综合服务平台在功能可用性、流程完整性与管理协同性方面的设计目标。研究结果可为高校学习空间数字化治理提供可落地的系统实现参考。

关键词：学习空间；实时状态；预约签到；智慧校园；协作服务

Abstract

With the continuous development of smart campus initiatives, learning space management in universities is shifting from static management to dynamic, refined, and service-oriented operations. To address common problems in existing campus systems, such as fragmented functions, untimely information updates, disconnected reservation and usage processes, and incomplete feedback loops, this thesis designs and implements a real-time-status-based integrated service platform for campus learning spaces.

The system adopts a front-end and back-end separated architecture. The front end is built with Vue 3, Pinia, and Vue Router to provide interactive interfaces for multiple roles, while the back end is developed with Spring Boot, MyBatis-Plus, MySQL, and Redis for business logic processing and data persistence. WebSocket is introduced to support real-time status updates.

Centered on the full lifecycle of learning space usage, the platform establishes a closed-loop workflow of "query-reservation-check-in-usage-feedback-governance". The user side supports building and classroom browsing, course occupancy viewing, learning space reservation, QR-code-based check-in and verification, classroom reviews, facility repair reporting, collaboration team functions, study plan management, resource marketplace, and campus navigation. The admin side supports repair ticket management, review moderation, marketplace audit, classroom and course management, building management, reservation rule configuration, and scan-device management.

Based on requirement analysis, this thesis presents the overall architecture, functional module decomposition, and database design strategy, and explains major implementation approaches through key business workflows. Test results show that the system can stably support core scenarios and meet the design goals in terms of functional availability, process integrity, and management collaboration. The research provides a practical reference for the digital governance of campus learning spaces.

Keywords: learning space; real-time status; reservation and check-in; smart campus; collaborative service

图目录

图 2.1 WebSocket 通信时序图 17
图 2.2 系统技术架构与技术栈示意图 18
图 2.3 鉴权与权限控制流程图 20
图 3.1 核心业务流程 23
图 3.2 系统用例图 27
图 3.3 系统总体架构图 29
图 3.4 核心 E-R 图 31
图 4.1 学习空间预约流程图 36
图 4.2 二维码核销流程图 37
图 4.3 预约与核销时序图 38
图 4.4 评价审核流程图 39
图 4.5 报修工单流转流程图 40
图 4.6 协作组队与审核流程图 41
图 4.7 聊天消息实时同步时序图 42
图 4.8 学习计划与预约联动流程图 43
图 4.9 资源市场审核与交易流程图 44
图 6.1 系统成果与后续演进路线图 57

表目录

表 3.1 角色-权限矩阵 21
表 3.2 预约状态示例 30
表 5.1 预约与核销功能测试用例 49
表 5.2 评价与审核功能测试用例 49
表 5.3 报修工单功能测试用例 50
表 5.4 协作与聊天功能测试用例 50
表 5.5 学习计划与资源市场功能测试用例 51
表 5.6 性能测试结果统计表 51

目 录

摘 要 1
Abstract 2
图目录 3
表目录 4
目 录 5
第 1 章 绪论 10
1.1 研究背景 10
1.1.1 智慧校园建设推动学习空间管理升级 10
1.1.2 现有校园学习空间系统存在的问题 10
1.1.3 课题研究的必要性 11
1.2 国内外研究现状 11
1.2.1 国外研究现状 11
1.2.2 国内研究现状 11
1.2.3 研究评述 11
1.3 研究目标与研究内容 12
1.3.1 研究目标 12
1.3.2 研究内容 12
1.3.3 论文组织结构 12
1.4 本章小结 12
第 2 章 相关技术与理论基础 13
2.1 前端关键技术 13
2.1.1 Vue 3 组件化开发模式 13
2.1.2 Vue 3 响应式系统与数据驱动视图 13
2.1.3 Pinia 状态管理 14
2.1.4 Vue Router 与前端路由治理 14
2.1.5 Axios 与统一接口封装 14

2.2 后端关键技术 15
2.2.1 Spring Boot 与分层架构 15
2.2.2 MyBatis-Plus 数据访问框架 15
2.2.3 MySQL 事务与一致性保障 16
2.2.4 Redis 缓存与高频读优化 16
2.2.5 WebSocket 实时通信机制 17
2.2.6 Spring Boot 文件上传与多媒体数据处理 17
2.3 关键业务技术原理 18
2.3.1 “实时状态”的业务定义与数据融合思路 18
2.3.2 预约规则约束与冲突校验 19
2.3.3 二维码生成与扫码核销 19
2.3.4 评价审核与内容治理 19
2.3.5 报修工单机制与状态机 19
2.3.6 RBAC 权限控制 20
2.4 本章小结 20
第 3 章 业务需求分析 21
3.1 业务需求分析 21
3.1.1 角色划分与职责 21
3.1.2 典型业务场景与用户目标 22
3.1.3 核心业务流程 23
3.2 功能需求分析 23
3.2.1 用户端功能需求 23
3.2.2 管理端功能需求 26
3.2.3 关键用例列表 27
3.3 非功能需求分析 28
3.3.1 可用性与体验需求 28
3.3.2 实时性需求 28
3.3.3 性能需求 28

3.3.4 安全性需求 28
3.3.5 可维护性与可扩展性需求 28
3.4 系统总体设计 29
3.4.1 总体架构设计 29
3.4.2 功能模块划分 29
3.4.3 数据流与接口风格设计 30
3.4.4 关键状态模型 30
3.5 数据库总体设计 30
3.5.1 主要实体识别 30
3.5.2 E-R 关系说明 31
3.5.3 关键约束、索引与一致性设计 32
3.6 本章小结 33
第 4 章 系统详细设计与实现 34
4.1 系统实现总体说明 34
4.1.1 分层架构与模块边界 34
4.1.2 核心数据建模特征 35
4.2 用户端核心模块设计与实现 35
4.2.1 用户注册与登录模块 35
4.2.2 学习空间浏览与状态查询 35
4.2.3 预约与签到核销闭环 35
4.2.4 反馈评价与报修服务 38
4.2.5 协作组队、聊天与学习计划联动 40
4.2.6 资源市场与校园导航扩展服务 43
4.3 管理端模块设计与实现 44
4.3.1 管理端登录、菜单与概览统计（RBAC） 44
4.3.2 评价审核管理 45
4.3.3 报修工单管理与通知 45
4.3.4 资源市场审核 45

4.3.5 基础数据管理与预约限制配置 45
4.4 本章小结 46
第 5 章 系统测试与结果分析 47
5.1 测试目标与测试环境 47
5.1.1 测试目标 47
5.1.2 测试范围 47
5.1.3 测试环境 48
5.1.4 测试方法 48
5.2 功能测试 49
5.2.1 功能测试设计原则 49
5.2.2 预约与核销功能测试 49
5.2.3 评价与审核功能测试 49
5.2.4 报修工单功能测试 50
5.2.5 协作与聊天功能测试 50
5.2.6 学习计划与资源市场功能测试 51
5.3 性能测试与结果分析 51
5.3.1 性能测试场景与指标 51
5.3.2 性能测试结果 51
5.3.3 性能结果分析 52
5.4 安全性与兼容性测试 52
5.4.1 权限与鉴权测试 52
5.4.2 输入校验与异常处理测试 53
5.4.3 浏览器兼容与可用性测试 53
5.5 测试结论与问题分析 53
5.5.1 测试结论 53
5.5.2 发现问题与改进建议 53
5.6 本章小结 54
第 6 章 总结与展望 55

6.1 研究工作总结 55
6.2 研究创新点与特色 56
6.3 不足与问题分析 56
6.4 未来展望 57
6.5 本章小结 58
参考文献 59
致 谢 61

第1章 绪论

1.1 研究背景

1.1.1 智慧校园建设推动学习空间管理升级

AI 70%

近年来，教育数字化转型持续推进，智慧校园建设已成为高校信息化建设的重要方向[1]。与教学、科研、管理等业务系统相比，学习空间管理长期存在“系统支持弱、业务协同低、用户体验差”的共性问题。学生在实际学习过程中，不仅需要知道“是否有空教室”，还关心环境舒适度、设施可用性、空间位置、预约规则及协作学习支持等信息。传统以静态公告和人工管理为主的模式，已难以满足高校高频、多样化、实时化的学习空间服务需求。

AI 63%

随着移动互联网、物联网和实时通信技术的普及，高校具备了将学习空间服务从“信息发布”升级为“综合治理与协同服务”的技术条件。通过整合空间状态、课程占用、预约记录、用户反馈和管理流程，能够实现学习空间服务的全流程数字化，进而提升资源利用效率和师生使用体验。

1.1.2 现有校园学习空间系统存在的问题

AI 67%

当前多数高校相关系统仍以“空教室查询”或“单点预约”功能为主，存在以下不足：

（1）功能割裂明显。查询、预约、签到、反馈、报修等业务彼此独立，用户在不同流程之间切换成本高，无法形成完整使用链路。

（2）实时性不足。空间状态更新机制不完善，用户看到的信息与现场状态可能不一致，影响预约决策与到场体验。

（3）治理闭环不完整。用户评价与报修信息缺乏有效流转机制，管理端难以形成可追踪、可分析、可改进的治理路径。

（4）协作支持不足。对小组学习、共享计划、团队沟通等协作场景支持有限，难以满足复合型学习需求[2]。

以上问题表明，学习空间管理不应仅停留在“查询工具”层面，而应向“综合服务平台”演进。

1.1.3 课题研究的必要性

AI 70%

基于实时状态的校园学习空间综合服务平台能够有效连接学生端与管理端，形成以用户需求为导向、以数据驱动为支撑的服务体系。一方面，平台可通过预约规则约束、签到核销、实时反馈机制提升空间流转效率，减少资源闲置与重复占用；另一方面，平台可将评价、报修、审核、配置等管理行为纳入统一系统，实现服务质量持续优化。由此，本课题具有明确的实践价值与推广意义。

1.2 国内外研究现状

1.2.1 国外研究现状

AI 70%

国外高校在学习空间数字化管理方面起步较早，相关平台通常具备较强的系统整合能力与数据驱动能力[3][4]。部分高校已将教室占用、图书馆座位管理、空间导航、环境监测等功能统一到校园移动平台中，并通过可视化统计支持资源优化配置。研究重点逐步由“是否可用”扩展到“如何高效使用”和“如何提升服务体验”，在实时状态采集、多源数据融合、行为分析与个性化推荐方面形成了较成熟思路。

1.2.2 国内研究现状

AI 70%

国内高校近年来在智慧教室、空教室查询、图书馆预约等领域发展迅速，系统覆盖率不断提升[1][2][5][6][7][8]。但从整体上看，仍存在功能模块分散、系统联动不足、治理闭环不完整等问题。多数应用围绕“查询与预约”展开，对签到核销、评价反馈、报修流转、协作学习等后续环节支持较弱；同时，管理侧对用户行为数据的利用深度有限，难以形成持续改进机制。

1.2.3 研究评述

AI 70%

综合国内外现状可见，学习空间服务平台建设正在从“单功能应用”向“全流程协同平台”转变。现有研究与实践为本课题提供了技术基础，但在业务闭环构建、角色协同治理、模块化工程实现等方面仍有提升空间[3][9][10]。本文以真实校园场景为导向，重点解决查询、预约、签到、反馈、治理的衔接问题，强调系统可实现性与落地性。

1.3 研究目标与研究内容

1.3.1 研究目标

本文目标是设计并实现一个面向高校场景的学习空间综合服务平台，具体目标如下：

（1）实现学习空间实时信息展示，支持用户快速完成空间选择；

（2）实现预约与签到核销闭环，提升空间周转效率；

（3）实现评价与报修闭环，增强管理响应能力；

（4）实现协作组队与学习计划功能，支持多样化学习组织形式；

（5）实现管理端统一治理能力，提升系统可运营性与可维护性。

1.3.2 研究内容

围绕上述目标，本文主要开展以下工作：

（1）进行校园学习空间业务需求分析，明确角色、场景与流程；

（2）完成系统总体架构设计与模块划分，确定前后端技术方案；

（3）完成数据库实体关系设计与关键表结构设计；

（4）实现用户端与管理端核心功能，打通主要业务流程；

（5）完成系统测试并分析结果，评估系统达成效果。

1.3.3 论文组织结构

AI 62%

全文共六章。第 1 章为绪论，介绍研究背景、现状与目标；第 2 章介绍相关技术与理论基础；第 3 章进行需求分析与总体设计；第 4 章阐述系统详细设计与实现；第 5 章给出系统测试与结果分析；第 6 章总结全文并提出未来展望。

1.4 本章小结

本章从智慧校园建设需求出发，分析了校园学习空间管理中存在的关键问题，论证了本课题的研究必要性与实践价值，明确了研究目标、研究内容和论文组织结构，为后续章节展开奠定基础。

第2章 相关技术与理论基础

本章围绕系统实现所涉及的前端框架、后端框架、数据存储与中间件、实时通信与二维码核销等关键技术进行阐述，并从“技术原理—选型理由—本系统落地方式”的角度说明其适用性与实现要点，为第 4 章的系统实现提供理论与技术支撑。

2.1 前端关键技术

2.1.1 Vue 3 组件化开发模式

（1）技术原理

AI 70%

Vue 采用“组件”作为最小复用单元，将界面拆解为可组合的结构。组件内部包含模板、逻辑与样式，通过 Props、事件与插槽实现组件间数据与结构的传递。在复杂业务页面中，组件化有助于将“展示”和“交互逻辑”以模块边界隔离，从而降低单文件复杂度。

（2）选型理由

学习空间综合服务平台页面多、业务流程长、角色多样（用户端 + 管理端）。Vue 3 具备成熟生态与高开发效率，适合快速迭代和模块化扩展；配合按需加载与路由拆分，可有效控制前端包体与首屏加载时间。

（3）本系统落地方式

本系统在用户端采用“预约中心—教室详情—二维码核销—反馈/报修—协作—计划—资源市场—导航”等多页面结构，天然适配组件化开发；在管理端采用统一管理框架页面承载多管理入口，便于后续扩展治理功能。

2.1.2 Vue 3 响应式系统与数据驱动视图

（1）技术原理

AI 70%

Vue 3 的响应式系统通过依赖追踪实现“状态变化自动触发视图更新”。当响应式状态被读取时，会收集依赖；当状态被修改时，会触发相关依赖的重新计算与渲染。该机制能够减少手工 DOM 更新，并将界面一致性建立在状态一致性之上。

（2）在本系统中的典型场景

预约状态变化（待签到/已核销）、协作红点角标变化、评价列表刷新、报修工单状态变化等均可通过状态更新驱动界面变化，从而提升交互连贯性与可维护性。

2.1.3 Pinia 状态管理

（1）技术原理

AI 69%

在单页应用中，跨页面共享数据若仅依靠组件层层传递会导致“数据来源不清、维护成本高”。Pinia 以 Store 为核心，提供 state、getter、action 三类能力：state 保存共享状态，getter 派生计算数据，action 封装异步请求与业务操作，形成集中可控的数据流。

（2）选型理由

相比传统 Vuex，Pinia 更轻量、API 更贴近组合式 API，学习成本低、工程组织更清晰。对于本系统这类多模块应用，Pinia 能降低跨页面状态同步难度，提升开发效率。

（3）本系统落地方式

本系统存在登录态与用户信息、协作消息提醒、预约跳转携带目标地点等跨页面共享需求，适合使用 Pinia 统一管理，避免“页面间隐式耦合”。

2.1.4 Vue Router 与前端路由治理

（1）技术原理

AI 70%

Vue Router 通过 History API 实现 URL 与页面组件映射，使应用在不刷新页面的情况下完成视图切换。路由不仅负责导航，还可承载路由参数（如 buildingId、roomId）、页面级懒加载以及路由守卫（鉴权控制）等能力。

（2）本系统落地方式

系统路由覆盖用户端与管理端入口：用户端包含首页、预约、反馈、协作、导航与个人中心等页面；管理端包含后台登录与管理页面集合。通过页面懒加载可降低首屏资源加载压力，提升移动端体验。

2.1.5 Axios 与统一接口封装

（1）技术原理

Axios 提供统一的 HTTP 请求能力，并支持请求/响应拦截器机制。拦截器可用于注入鉴权信息（如 token）、统一错误处理（如网络异常、接口返回异常码）以及统一解析返回结构。

（2）本系统落地方式

前端对后端接口采用统一封装：

1）统一配置 API Base URL，保证多环境切换便利；

2）通过拦截器集中处理登录过期、权限不足等异常提示；

3）对管理端请求可按后端约定注入管理端 token（例如通过请求头传递），从而实现用户端与管理端鉴权策略隔离。

2.2 后端关键技术

2.2.1 Spring Boot 与分层架构

（1）技术原理

Spring Boot 提供自动配置、依赖管理与约定优于配置等能力，适合快速构建可运行的 Web 服务。分层架构中：

1）Controller 层负责参数接收、鉴权入口与返回结果封装；

2）Service 层负责业务流程编排与规则校验；

3）Mapper 层负责数据库访问与数据对象映射。

分层架构能显著降低“接口逻辑与业务逻辑、业务逻辑与数据访问”之间的耦合。

（2）本系统落地方式

本系统以 REST 接口形式对外提供服务，后端统一返回结构（便于前端统一处理），并将预约规则校验、二维码核销、审核流转等核心逻辑封装在 Service 层，提升复用性与可测试性。

2.2.2 MyBatis-Plus 数据访问框架

（1）技术原理

MyBatis-Plus 在 MyBatis 的基础上提供通用 CRUD、条件构造器、分页等能力，减少重复 SQL 编写。其实体映射机制能够将表结构与实体对象对应，配合全局配置可实现逻辑删除字段的统一管理。

（2）选型理由与落地方式

本系统数据表数量较多且增删改查场景丰富（预约、评价、报修、协作、资源市场等），使用 MyBatis-Plus 能显著提升开发效率，并统一数据层编码风格。

2.2.3 MySQL 事务与一致性保障

（1）技术原理

MySQL 支持 ACID 事务，可用于保证多步骤更新的一致性，例如“创建预约记录—写入时间槽占用—生成核销状态”等场景。通过合理的索引设计，可在保证一致性的同时提升查询性能。

（2）本系统落地方式

系统核心数据（用户、空间资源、预约、评价、报修、协作、交易等）均采用 MySQL 存储。预约、核销、审核与工单流转等关键流程需要保证状态更新的一致性与可追溯性，适合使用关系型数据库承载。

2.2.4 Redis 缓存与高频读优化

（1）技术原理

Redis 基于内存存储，适合存放访问频率高、更新频率相对可控的数据（热数据），能够显著降低数据库压力。常见使用方式包括：

1）缓存查询结果（如空间列表、预约余量、二维码状态短期缓存）；

2）存放临时状态（如短期 token、验证码、设备配对状态等）；

3）作为分布式协调组件。

（2）本系统落地方式

系统在预约与二维码状态查询等高频场景引入 Redis，提升响应速度与稳定性，并为后续的高并发控制策略预留扩展空间，系统在缓存与平台层协同设计上与高校智慧校园云平台建设思路一致[11]。

2.2.5 WebSocket 实时通信机制

（1）技术原理

WebSocket 在浏览器与服务端之间建立长连接，支持双向消息推送。与轮询相比，WebSocket 能减少无效请求并降低状态变更传递延迟，适合实时通知与状态更新。

（2）本系统落地方式

系统在“预约状态变化提示”“协作消息提醒”“管理端处理结果通知”等场景中具备实时推送需求，引入 WebSocket 能提升用户端体验与管理协同效率。

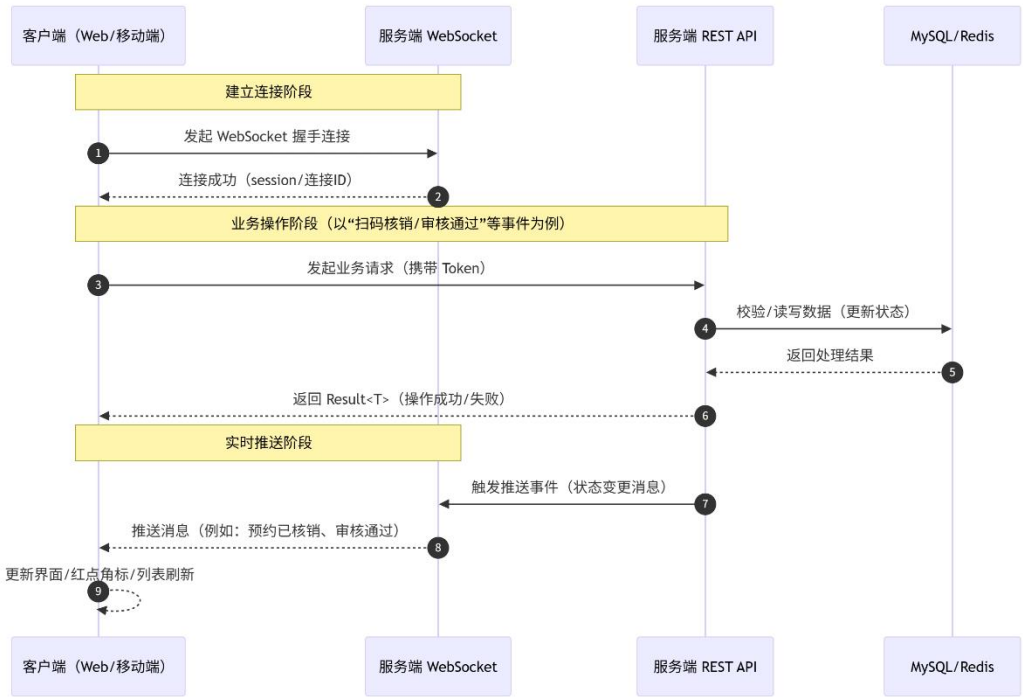

图 2.1 WebSocket 通信时序图

2.2.6 Spring Boot 文件上传与多媒体数据处理

（1）技术原理

AI 69%

Spring Boot 通过 Multipart 机制支持文件上传，可在服务端进行文件大小限制、类型校验与存储路径管理。对报修图片等多媒体数据，通常采用“文件存储 + 数据库保存元信息”的方式：数据库保存文件 URL/路径、关联工单 ID、上传时间等，文件本体存放于本地或对象存储。

（2）本系统落地方式

系统支持报修图片上传，并在后端配置文件上传大小限制，保证在常见网络环境下上传可靠且不影响服务稳定性。

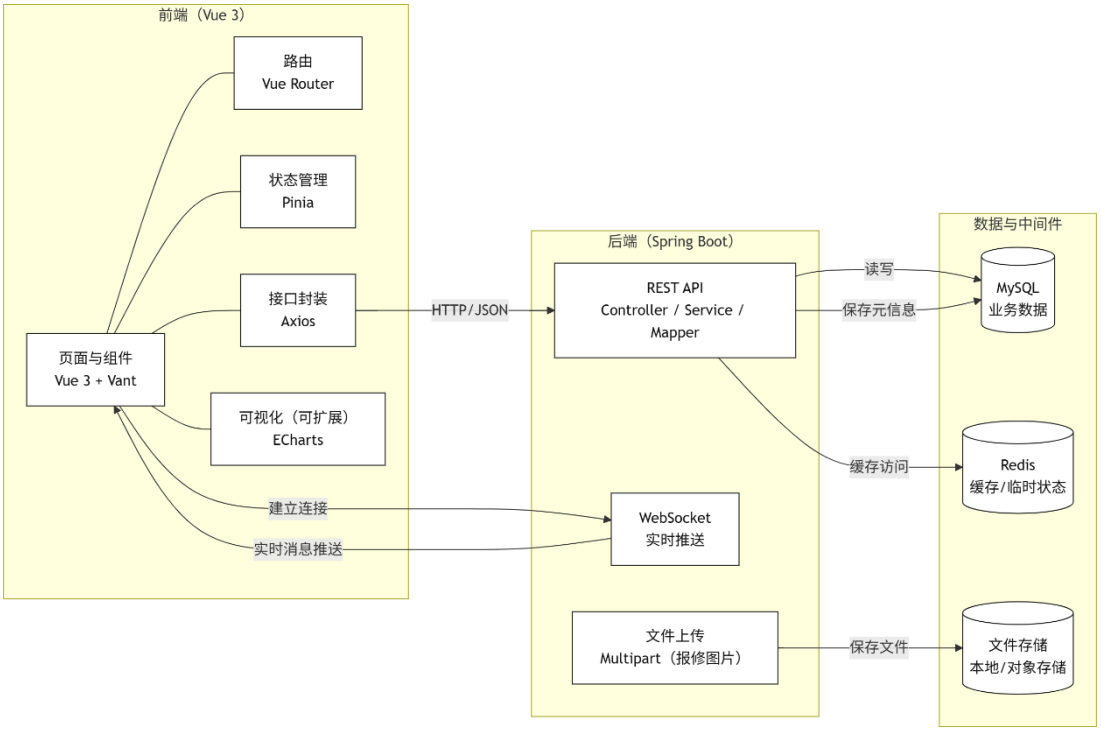

图 2.2 系统技术架构与技术栈示意图

如图 2.2 所示，系统采用前后端分离架构，后端通过 MySQL 与 Redis 完成数据存储与缓存支撑。

2.3 关键业务技术原理

2.3.1 “实时状态”的业务定义与数据融合思路

学习空间的“实时状态”不是单一数据源，而是多源信息融合后的对外展示结果。典型数据源包括：

（1）课程排课：决定教室在某时间段是否被教学占用；

（2）预约记录：决定教室/座位在某时间段是否被用户预约；

（3）核销状态：反映预约是否已到场确认，可用于辅助判断“占而不到”等问题；

（4）报修状态：若空间设施报修中，可提示用户选择其他空间或给出风险提醒。

在工程实现上，可将状态展示抽象为“按时间段的占用判定”，由后端统一计算后下发前端，避免前端分别调用多接口并在页面层拼装导致的不一致风险。后续可引入人流密度识别数据作为实时状态融合输入，提升状态感知精度[12]。

2.3.2 预约规则约束与冲突校验

预约模块需要满足“规则可配置、校验可复用、结果可解释”。常见约束：

（1）次数约束：每周预约次数上限；

（2）时长约束：单次预约最长时长与连续时段上限；

（3）冲突约束：课程占用冲突、已预约冲突、时间槽不可用等；

（4）状态约束：已核销/已取消/已过期等状态不可重复操作。

系统应在服务端集中完成校验并返回明确错误信息，前端将错误信息以用户可理解方式提示，从而提升可用性与系统可信度。

2.3.3 二维码生成与扫码核销

（1）技术原理

二维码本质是对一段信息的编码，扫码核销需要解决两个核心问题：二维码内容如何与预约记录唯一绑定；扫码提交后如何进行有效性校验与状态更新。

（2）典型校验点

校验预约是否存在、是否属于有效时间窗口、是否已核销、是否被取消/过期等，并在核销成功后更新预约状态，返回可追溯结果，防止重复核销或伪造二维码。

2.3.4 评价审核与内容治理

AI 70%

公开评价既能为后续用户提供参考，也存在不当内容风险，因此需建立审核机制。典型策略为：用户提交评价后进入待审核状态；审核通过后对外公开；审核拒绝则不公开但保留记录（便于申诉与追溯）。该机制体现了平台从“工具”走向“治理”的能力建设。

2.3.5 报修工单机制与状态机

AI 60%

报修业务可抽象为有限状态机：待处理 -> 处理中 -> 已完成（也可扩展已驳回/已关闭）。通过状态机可将“用户发现问题—管理响应—处理完成”形成闭环，

并沉淀可统计数据（故障类型、处理时长、集中故障点位等），为后续优化提供依据。

2.3.6 RBAC 权限控制

AI 70%

系统存在多角色、多入口的权限需求。RBAC（Role-Based Access Control）通过“用户—角色—权限”关系实现访问控制：用户被分配角色，角色拥有权限集合，接口或菜单依据权限开放。

本系统将用户端与管理端进行隔离：用户端围绕学习服务；管理端围绕审核、工单与配置等治理能力。通过鉴权头与菜单权限控制，可降低越权风险并提升后台操作安全性。如图 2.2 所示，系统通过登录鉴权、权限校验与异常分支处理实现访问控制。

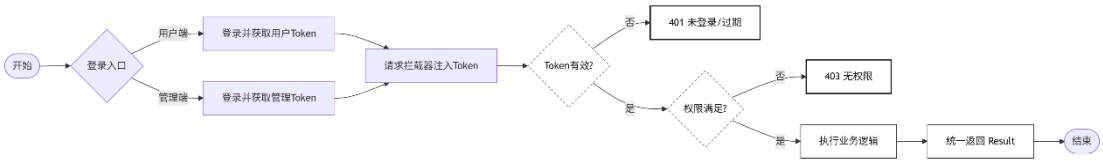

图 2.3 鉴权与权限控制流程图

2.4 本章小结

本章围绕平台建设所需的技术体系，分别从前端框架、后端框架、数据存储与中间件、实时通信机制以及关键业务技术原理等方面进行了系统梳理。通过对 Vue 3、Pinia、Vue Router、Axios、Spring Boot、MyBatis-Plus、MySQL、Redis 与 WebSocket 等技术的原理分析与选型论证，明确了系统在工程化实现中的技术路径与组件协同关系。

AI 70%

从应用适配性看，上述技术组合能够较好匹配校园学习空间综合服务平台“多模块协同、状态实时更新、流程闭环治理”的业务特征：前端技术栈支撑页面快速迭代与交互一致性，后端分层架构支撑复杂业务规则沉淀与复用，关系型数据库与缓存协同保障数据一致性与高频访问性能，实时通信机制满足预约核销与协作消息的低延迟反馈需求。

综上，本章完成了从“技术原理”到“场景落地”的论证闭环，为第 3 章的需求分析与总体设计提供了方法依据，也为第 4 章详细设计与实现奠定了可执行的技术基础。

第3章 业务需求分析

本章在任务书与开题报告的目标约束下，结合系统当前已实现功能，完成角色与业务场景梳理、功能与非功能需求归纳，并给出系统总体架构、模块划分与数据库总体设计，为第 4 章的详细设计与实现提供依据。

3.1 业务需求分析

3.1.1 角色划分与职责

系统面向校园学习空间全流程服务，按照“前台服务 + 后台治理”的思路划分多角色协同，由表 3.1 可见，不同角色在系统中承担的职责边界明确，为后续功能模块划分与权限控制提供了依据。

表 3.1 角色-权限矩阵

角色	主要权限	典型操作
学生用户	学习空间查询、预约、二维码查看、扫码核销、评价、报修、协作、学习计划、资源市场、校园导航	查询教室占用、提交预约、核销签到、提交评价/报修、加入小组、创建计划、发起交易
教师用户	与学生端类似的前台使用权限（可按系统配置扩展）	查询空间、预约研讨室、参与协作讨论、发布学习资源
管理员	后台登录、菜单访问、基础数据管理、预约规则配置、扫码设备管理、概览统计	维护教室/课程/教学楼、调整预约上限、管理扫码设备
评价审核员	评价审核权限	审核评价通过/驳回、维护评价内容质量
资源审核员	资源市场审核权限	审核资源发布内容、记录审核意见
维修人员	报修工单处理权限	查看工单、更新处理状态、填写处理备注
扫码设备端	核销执行权限（受授权设备限制）	扫码提交核销、回传设备标识

（1）学生用户（普通用户）

主要任务：浏览教学楼/教室信息、查询占用与可用时段、发起预约并完成签到核销、提交教室评价与报修、参与协作小组与聊天、维护学习计划、参与资源市场交易、使用校园导航到达目标点位等。

（2）教师用户（扩展角色，可与学生同属前台用户体系）

主要任务：除学生能力外，更偏向于协作学习组织与固定空间使用需求（实际功能以系统实现为准）。在论文描述中可作为“前台用户的一类角色”，用于说明平台支持多类型用户。

（3）管理员（后台治理角色）

主要任务：后台登录与菜单访问、教学楼/教室/课程基础数据维护、预约规则（周上限、单次时长等）配置、扫码设备管理、平台概览统计（若已实现）等。

（4）评价审核员（内容治理角色）

主要任务：审核用户评价内容，控制公开展示范围，屏蔽不当言论，保证公共评价可参考性。

（5）资源市场审核员（内容治理角色）

主要任务：对资源市场发布内容进行审核（通过/拒绝），保障平台内容合规。

（6）维修/运维人员（工单处理角色）

主要任务：查看报修工单，推进工单状态（待处理/处理中/已完成），形成设备治理闭环。

3.1.2 典型业务场景与用户目标

为确保平台功能贴近校园真实使用，本研究将业务归纳为以下典型场景：

（1）临时自习：学生在空闲时间快速查找可用教室/座位并预约，到场签到后使用。

（2）小组讨论：协作小组需要在固定时间集中学习，要求空间可预约且支持多人协作，并可在计划中关联研讨室预约。

（3）到场导航：用户预约成功后需要快速到达目标地点，希望系统提供路径规划或直线导航降级方案。

（4）使用后反馈：用户使用教室后对环境、插座、噪音等进行评价；若发现设备故障则提交报修并跟踪处理进度。

（5）运营治理：管理员/审核/维修人员在后台对评价、报修、资源发布等进行治理，保障平台秩序并提升服务质量。

3.1.3 核心业务流程

图 3.1 展示了用户端与管理端的协同关系，说明本系统不仅关注资源使用效率，也强调反馈治理与持续优化能力。

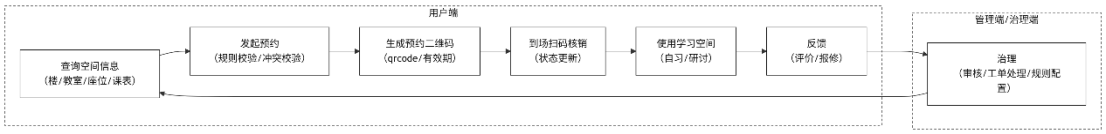

图 3.1 核心业务流程

（1）学习空间使用闭环

查询空间信息 → 查询时段占用 → 选择连续时段 → 提交预约（规则/冲突校验） → 生成二维码 → 到场扫码核销 → 使用完成 → 评价/报修 → 后台治理。

（2）内容治理闭环

用户提交评价/资源 → 后台审核 → 通过后公开展示/交易可见；拒绝则不公开但可留痕追溯。

（3）设施治理闭环

用户提交报修（图文） → 后台接单 → 状态推进 → 完成闭环 → 数据沉淀（类型、频次、处理时长）。

3.2 功能需求分析

3.2.1 用户端功能需求

（1）首页与基础浏览模块

功能点：

1）提供平台主要服务入口；

2）教学楼列表与详情展示；

3）按教学楼查看教室列表与教室详情；

4）楼宇导航与导航详情展示；

5）教学楼课表/排课信息展示。

需求目标：降低用户信息获取成本，支持“先查再约”的决策路径。

（2）学习空间预约模块（教室/图书馆座位）

功能点：

1）预约中心按日期切换，支持资源分类（教室/座位等）；

2）教室详情展示时段占用情况与可预约时间段；

3）支持连续时段预约；

4）预约规则读取并限制：每周预约次数上限、单次预约最长时长；

5）个人预约记录查询、状态展示与跳转；

6）预约二维码生成与展示；

7）预约签到入口与核销状态查询；

8）从“我的预约（待签到）”一键跳转校园导航并带入目标地点。

需求目标：形成“预约—到场确认—使用”的闭环，提高空间周转效率。

（3）评价与反馈模块

功能点：

1）待评价记录查询；

2）教室评价提交（星级/文字等）；

3）个人评价记录查看与删除；

4）教室详情页展示审核通过的评价；

5）查看全部评价页面；

6）用户提交报修（支持图片）；

7）个人报修记录查看。

需求目标：构建“使用后反馈”链路，提升信息透明度与服务质量。

（4）协作组队与沟通模块

功能点：

1）协作广场浏览活跃小组；

2）查看小组详情与成员列表；

3）发起协作小组、加入/退出、删除自己创建的小组；

4）加入申请提交与组长审核（通过/拒绝）；

5）申请结果查询与待处理申请查询；

6）协作红点角标提醒；

7）小组聊天；

8）消息中心查看协作相关消息；

9）进入研讨室协作页面。

需求目标：支持小组学习组织与沟通协作，将“空间”与“人”连接起来。

（5）学习计划模块

功能点：

1）共享学习计划页展示；

2）按协作小组查询学习计划；

3）按预约查询关联学习计划；

4）创建“学习计划＋教室预约”联动记录；

5）更新学习计划；

6）计划中关联研讨室、日期、时间段与关键时间节点。

需求目标：强化协作学习的计划性，将“目标—时间—空间”贯通。

（6）课程表与我的课程模块

功能点：

1）按教学楼查看课表（按日/周/月）；

2）教室课表查询与时段占用状态查询；

3）按楼层/关键字/时间段筛选日课表；

4）我的课程按日/周查看；

5）课程星标、备注、课前提醒时间配置。

需求目标：将课程占用作为空间状态的重要数据源，避免预约与教学冲突。

（7）资源市场与校园学分模块

功能点：

1）资源市场广场浏览与筛选；

2）发布学习资源/二手资源（标题、描述、类别、价格、图片、推荐交易地点）；

3）发起交易意向并填写留言与线下交接地点；

4）交易双方确认完成；

5）个人发布记录与交易记录；

6）个人资源管理维护；

需求目标：扩展平台服务边界，增强用户活跃度与平台价值。

（8）校园导航模块

功能点：

1）校园地图浏览（教学楼+公共场馆点位）；

2）按名称/编号/地址搜索目标点位；

3）定位“我的位置”并展示与目标距离；

4）一键规划步行路线；

5）预约场景携带目标名称自动选中并尝试规划路线。

需求目标：解决“约到但找不到”的最后一公里问题。

3.2.2 管理端功能需求

（1）后台认证与菜单权限

支持管理员登录；根据权限返回后台菜单；通过菜单与接口双重限制避免越权。

（2）报修工单管理

支持工单列表查询、状态更新、处理过程记录与筛选；保障设施治理闭环可追踪。

（3）评价审核

支持评价列表、通过/拒绝操作；通过后允许在用户端公开展示，拒绝则不公开。

（4）资源市场审核

支持资源发布内容审核（通过/拒绝）；通过后对外可见并参与交易流程。

（5）基础数据管理

支持教室管理、课程管理、教学楼管理；支持导入/维护。

（6）预约限制配置

支持每周预约次数上限、单次最长时长等规则配置；当库表无配置时使用后端默认值兜底。

（7）扫码设备管理

支持生成设备配对 token、设备端扫码授权绑定 UID、设备列表查看、启用/停用、删除；支撑预约扫码核销场景。

3.2.3 关键用例列表

为进一步明确角色与功能交互关系，本文将关键用例进行图形化表达，如图 3.2 所示，系统用例覆盖用户端学习活动与管理端治理流程，体现了平台一体化服务特征。

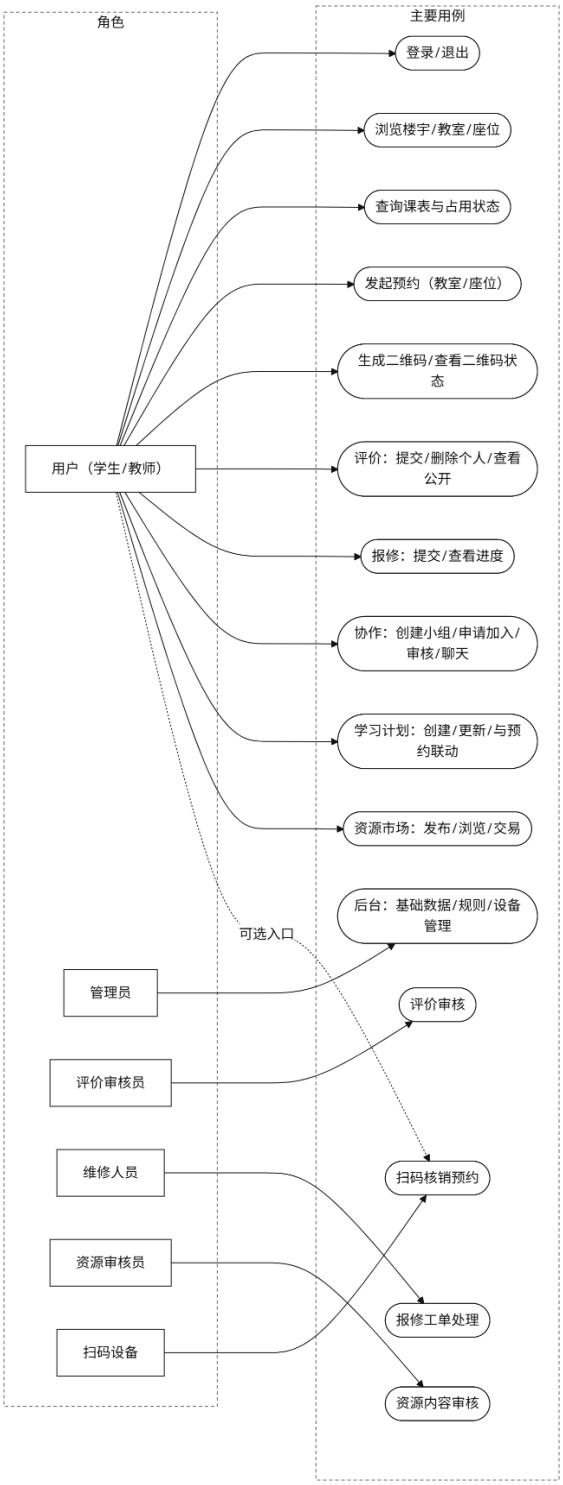

图 3.2 系统用例图

3.3 非功能需求分析

3.3.1 可用性与体验需求

AI 67%

（1）移动端友好：页面布局适配手机端；关键操作尽量在 3 步内完成。

（2）交互一致性：列表筛选、空态、错误提示、加载状态保持一致。

（3）可理解性：预约失败必须返回可解释原因（超限/冲突/不可用）。

在多终端交互中持续优化用户体验是提升平台可用性的关键[13]。

3.3.2 实时性需求

（1）预约状态与核销结果需要快速反馈；

（2）协作消息与红点提醒应及时更新；

（3）关键状态变化可通过 WebSocket 推送。

3.3.3 性能需求

AI 56%

（1）常用查询接口：在一般网络条件下应保持较低延迟。

（2）高峰预约场景：并发预约请求增多时，系统应保证规则校验正确、数据一致，并保持可接受的错误率与响应时间。

（3）缓存策略：对热点数据（教室列表、占用信息、二维码状态）采用 Redis 缓存降低数据库压力。

3.3.4 安全性需求

AI 69%

（1）鉴权：用户端与管理端 token 隔离；接口需校验登录态与权限。

（2）输入校验：对关键参数（时间段、资源 ID、图片上传等）进行合法性校验。

（3）数据保护：密码等敏感信息需安全存储，管理端操作具备可追踪性。

3.3.5 可维护性与可扩展性需求

（1）前后端分离与接口规范：统一返回结构 `Result<T>`，便于前端统一处理；

（2）模块化：预约、反馈、协作、计划、市场、管理端等模块相对独立；

（3）配置化：预约规则通过配置表管理，支持无配置兜底默认值。

3.4 系统总体设计

3.4.1 总体架构设计

AI 70%

系统采用前后端分离架构：前端负责页面展示与交互；后端负责业务编排、规则校验、状态计算、数据持久化与消息推送；MySQL 作为主数据存储；Redis 作为缓存与临时状态存储；WebSocket 作为实时消息通道。结合功能与非功能需求，系统采用前后端分离架构，总体结构如图 3.3 所示，表明系统通过 REST 接口完成业务交互，通过数据库与缓存协同支撑核心流程，并以实时通信机制提升用户体验。

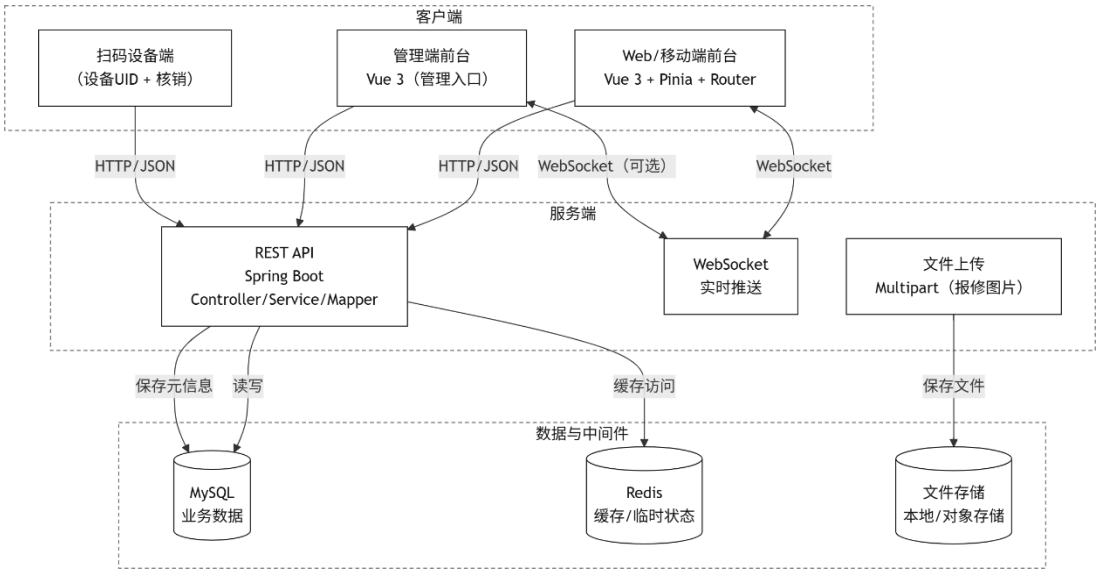

图 3.3 系统总体架构图

3.4.2 功能模块划分

为保证开发与维护效率，系统按业务域划分模块，相关移动应用开发实践也表明，模块化组织有助于提高迭代效率与可维护性[14]。

（1）空间与课表模块：教学楼、教室、课表查询、占用状态展示；

（2）预约与核销模块：预约创建、规则校验、二维码生成、扫码核销、状态查询；

（3）反馈与治理模块：评价提交/展示/审核、报修提交/工单处理；

（4）协作与消息模块：组队、申请审核、消息中心、聊天、协作页面；

（5）学习计划模块：计划创建/更新、与小组/预约联动；

（6）资源市场模块：资源发布/审核/交易；

（7）校园导航模块：点位搜索、定位与路线规划；

（8）后台管理模块：基础数据、规则配置、扫码设备管理等。

3.4.3 数据流与接口风格设计

AI 69%

（1）统一返回结构：后端对外统一返回 `Result<T>`，包含 code、message 与 data，便于前端统一处理成功/失败。

（2）接口分域：按业务域组织接口（如预约、评价、报修、协作等），降低耦合。

（3）状态驱动：重要状态在后端集中计算与校验（例如预约冲突与超限），前端以展示与交互为主，避免前端自行拼装造成不一致。

3.4.4 关键状态模型

表 3.2 定义了预约业务中的关键状态与可执行操作，为后续实现中的状态流转控制提供依据。

表 3.2 预约状态示例

状态	含义	允许操作
待签到	预约成功但未核销	生成二维码、核销、取消
已核销	已到场确认	查看详情（一般不可取消）
已取消	用户取消/管理员取消	仅查看
已过期	超过时间窗口未核销	仅查看（可作为违约统计扩展）

3.5 数据库总体设计

3.5.1 主要实体识别

结合系统功能模块，数据库核心实体包括：

用户（User）、角色/权限（Role/Permission，可选）、教学楼（Building）、教室（Classroom）、图书馆与座位（Library/Seat）、课程（Course）、课程表/排课（CourseSchedule）、时间槽（TimeSlot）、预约（Reservation）、核销/扫码记录（Scan/Verification，可选）、评价（Review）、评价审核（ReviewAudit，可选/或字段承载）、报修工单（RepairTicket）、工单处理记录（RepairLog，可选）、协作小组（Team）、小组成员（TeamMember）、入组申请（TeamJoinRequest）、

小组消息/聊天（TeamMessage/Chat）、学习计划（StudyPlan）、计划节点（PlanMilestone，可选）、资源条目（MarketItem）、交易意向/交易记录（Trade/Deal）、扫码设备（ScanDevice）、扫码授权（ScanDeviceAuth）。

3.5.2 E-R 关系说明

根据实体识别结果与业务关系分析，可得到系统核心数据关系模型，如图 3.4 所示，展示了用户、空间资源、预约、反馈、协作、计划与交易等实体间的关键关联关系，为表结构设计奠定基础。

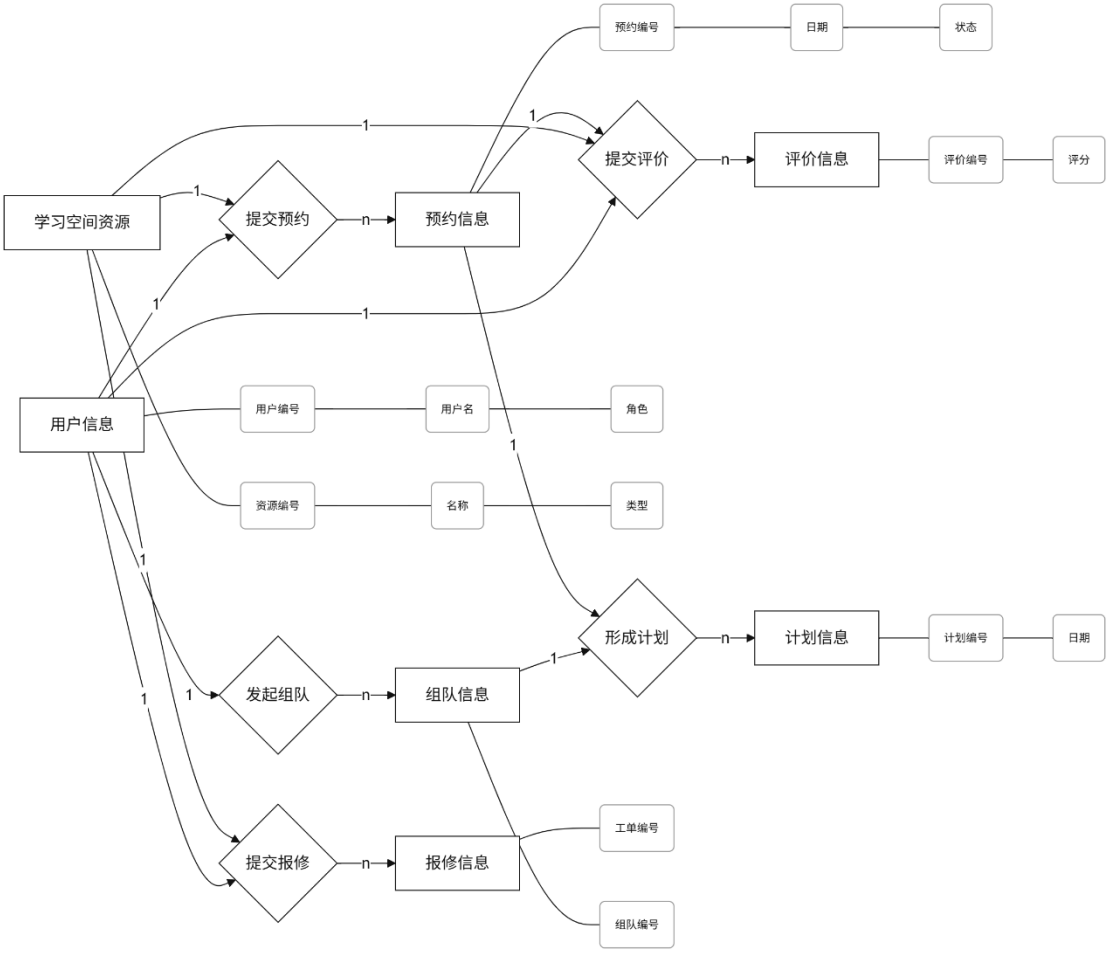

图 3.4 核心 E-R 图

（1）用户 — 预约：一名用户可创建多条预约记录（1:N）。

（2）学习空间资源 — 预约：一个学习空间资源在不同时间槽可对应多条预约（1:N）。

（3）用户 — 评价：用户可对预约过的学习空间提交多条评价（1:N），评价公开展示受审核状态控制。

AI 67%

（4）学习空间资源 — 评价：一个学习空间资源可被多名用户评价（1:N），评价结果影响资源热度与推荐。

（5）用户 — 报修工单：用户可针对学习空间资源提交多条报修工单（1:N），工单状态推进形成服务闭环。

AI 69%

（6）学习空间资源 — 报修工单：一个学习空间资源可关联多条报修工单（1:N），用于记录设施故障情况。

（7）用户 — 协作组队：一名用户可发起或加入多个协作小组（1:N），小组与用户间通过成员表维护多对多关系（N:M）。

AI 69%

（8）协作组队 — 学习计划：一个协作小组可制定多条学习计划（1:N），用于规划学习时间与目标。

（9）预约 — 学习计划：学习计划可关联预约记录，实现“计划 - 空间”联动（1:1 或 1:N，按实现逻辑选择），为用户提供学习路径与场地安排的统一视图。

3.5.3 关键约束、索引与一致性设计

AI 70%

（1）统一资源建模（教室/座位）

`reservation`、`review`、`repair` 等核心业务表均采用 `resource_type + classroom_id/library_seat_id` 的统一资源建模模式，即通过资源类型字段区分“教室/图书馆座位”等对象，并结合对应资源主键建立业务关联。

（2）唯一约束防止脏数据

例如 `classroom` 的 `uk_classroom_room (building_id,room_number)` 防止同楼同号重复；`library_seat` 的 `uk_library_seat (library_id,floor,row_num,col_num)` 保证座位坐标唯一；`team_member` 的 `uk_team_user` 防止重复入组。

（3）高频查询索引

`reservation` 的 `idx_resource` 与 `idx_reservation_date` 支撑“按资源+日期查时段占用”；`review/repair` 的 `idx_resource` 支撑“按资源查评价/工单”；`team_message` 的 `idx_team_time_id` 支撑聊天分页。

（4）状态字段的枚举化与可追溯

核心流程（预约、审核、工单）均采用 TINYINT 状态字段，并保留 `create_time/update_time` 与 `deleted` 逻辑删除字段，便于追溯与统计。

（5）一致性与幂等

AI 70%

聊天表通过 `client_msg_id` + 唯一键实现幂等；预约核销链路通过记录 `qrcode_scan_time/device_uid` 等字段提升可追溯性。对于“创建预约—生成二维码—核销更新状态”等关键更新，应采用事务与必要的重复提交防护策略，避免状态错乱。

3.6 本章小结

AI 67%

本章围绕“基于实时状态的校园学习空间综合服务平台”建设目标，系统完成了需求分析与总体设计工作。首先，通过角色划分与典型场景梳理，明确了用户端服务与管理端治理的协同关系；其次，围绕预约核销、反馈治理、协作学习、资源市场与后台管理等核心业务，形成了较完整的功能需求集合，并从可用性、实时性、性能、安全性、可维护性与可扩展性等维度给出了非功能约束。

AI 88%

在总体设计层面，本章给出了前后端分离架构、模块化分域方案、统一接口风格与关键状态模型，确保系统在工程实现中具备清晰边界与一致语义。在数据设计层面，通过核心实体识别、E-R 关系定义、关键表结构与索引约束说明，构建了能够支撑多业务闭环的数据基础，并通过统一资源建模与状态字段设计为后续扩展预留空间。

综上，本章实现了从“业务问题”到“系统方案”的结构化转化，为第 4 章模块级详细设计、接口实现与数据库落地提供了明确依据，同时也为第 5 章测试验证中的指标与场景设计奠定了需求基线。

第4章 系统详细设计与实现

本章在第 3 章总体设计的基础上，结合系统当前代码实现与真实数据库表结构，围绕用户端与管理端两条主线，对关键业务模块的设计思路、核心流程、接口实现与数据表映射进行说明。本文系统采用前后端分离架构，后端以 Spring Boot + MyBatis-Plus 为主，统一提供 REST API，并在小组聊天等场景引入 WebSocket 实时推送；数据存储以 MySQL 为主，Redis 用于缓存/临时状态支撑高频读取。

4.1 系统实现总体说明

AI 69%

本节从系统落地实现视角，说明平台在工程结构、业务分层与数据组织上的总体方案。为兼顾多业务并发、实时反馈与后台治理需求，系统在实现上遵循“统一接口规范、统一状态语义、统一资源抽象”的设计原则：接口层保证调用一致性，业务层保证规则可复用与可演进，数据层保证实体关系稳定与可追踪。通过该实现路径，系统能够在不牺牲可维护性的前提下支撑预约核销、反馈治理、协作通信、市场审核等多条业务链路并行运行。系统采用模块化迭代实现路径，符合敏捷开发在教育软件场景中的实践趋势[15]。

4.1.1 分层架构与模块边界

AI 69%

后端采用 Controller-Service-Mapper 分层结构：Controller 层负责 HTTP 接口与参数校验入口；Service 层负责业务规则、状态流转与流程编排；Mapper 层负责数据库读写与必要的联表查询。该结构使得预约规则校验、二维码核销、评价审核、工单处理、协作聊天等复杂逻辑能够集中在业务层实现，便于复用与测试。

前端以 Vue 3 + Pinia + Vue Router 组织页面与状态管理，页面按业务域划分：预约中心、反馈评价、协作组队、学习计划、资源市场、校园导航、个人中心与后台管理入口等。前端通过 Axios 统一封装请求，后端统一返回 Result<T> 结构，降低前端错误处理复杂度。

4.1.2 核心数据建模特征

AI 66%

系统在“预约/评价/报修”等关键业务中采用统一资源模型，即通过`resource_type`+`classroom_id/library_seat_id`实现“教室/图书馆座位”两类资源的统一建模：

预约表：`reservation`

评价表：`review`（通用评价），以及`classroom_feedback`（用于反馈页的评价流转）

报修表：`repair`

统一资源模型的优势是减少表数量，避免为不同资源维护两套业务流程，并便于前端“按资源类型”统一展示与筛选。

4.2 用户端核心模块设计与实现

4.2.1 用户注册与登录模块

AI 66%

用户端以统一账号体系为入口，完成注册、登录与身份识别。系统采用同一用户表承载学生、教师及管理类账号，通过角色字段区分权限边界。登录后，前端保存会话信息并加载个人中心相关数据，为后续预约、反馈、协作等功能提供身份上下文。

4.2.2 学习空间浏览与状态查询

AI 63%

该模块实现“楼宇-教室-时段”三级查询能力。用户可先浏览教学楼与教室基础信息，再结合课程安排与预约占用查看目标时段可用性。系统通过标准时间段模型统一前后端时间粒度，避免页面端拼装多源数据造成状态不一致。

4.2.3 预约与签到核销闭环

AI 69%

用户端核心业务围绕“预约创建-二维码生成-扫码核销-状态更新”展开。系统支持教室与图书馆座位统一预约，并在后端执行规则校验（周上限、单次时长）与冲突校验（课程/已预约占用）。为直观说明预约业务在创建阶段的关键步骤与状态变化关系，本文将学习空间预约流程抽象为统一业务链路，如图4.1所示。

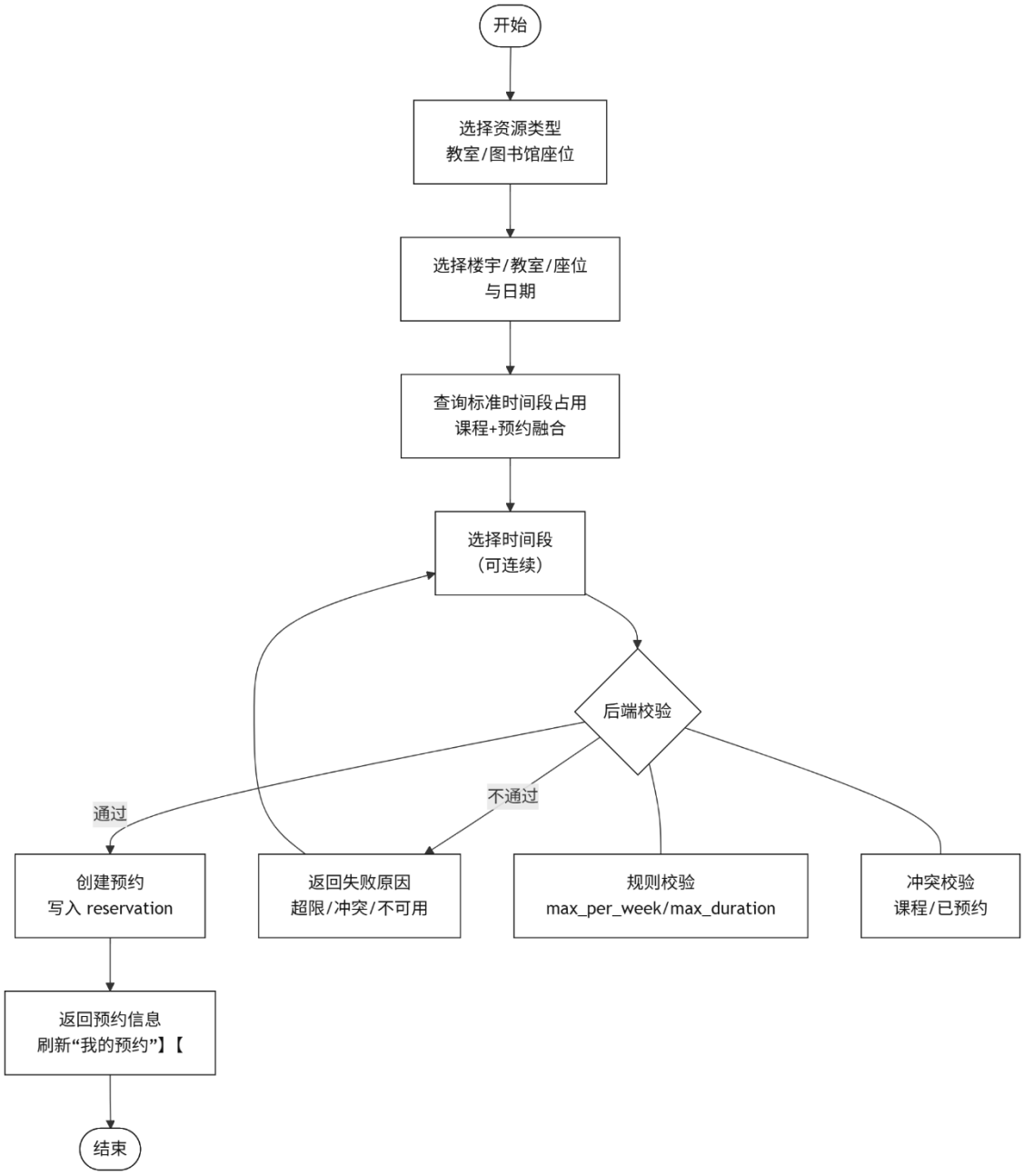

图 4.1 学习空间预约流程图

AI 70%

由图 4.1 可见，预约创建阶段已完成资源可用性与规则约束的双重校验，为后续核销环节提供可信前置条件。

核销环节采用跨设备扫码模式：用户端生成二维码并轮询状态，设备端执行扫码核销，服务端完成有效性判定与预约状态流转。通过该机制将线上预约与线下到场确认连接起来，有效降低“预约未到场”带来的资源浪费。在完成预约创建后，系统通过跨设备扫码机制实现到场确认，其核心核销流程如图 4.2 所示。

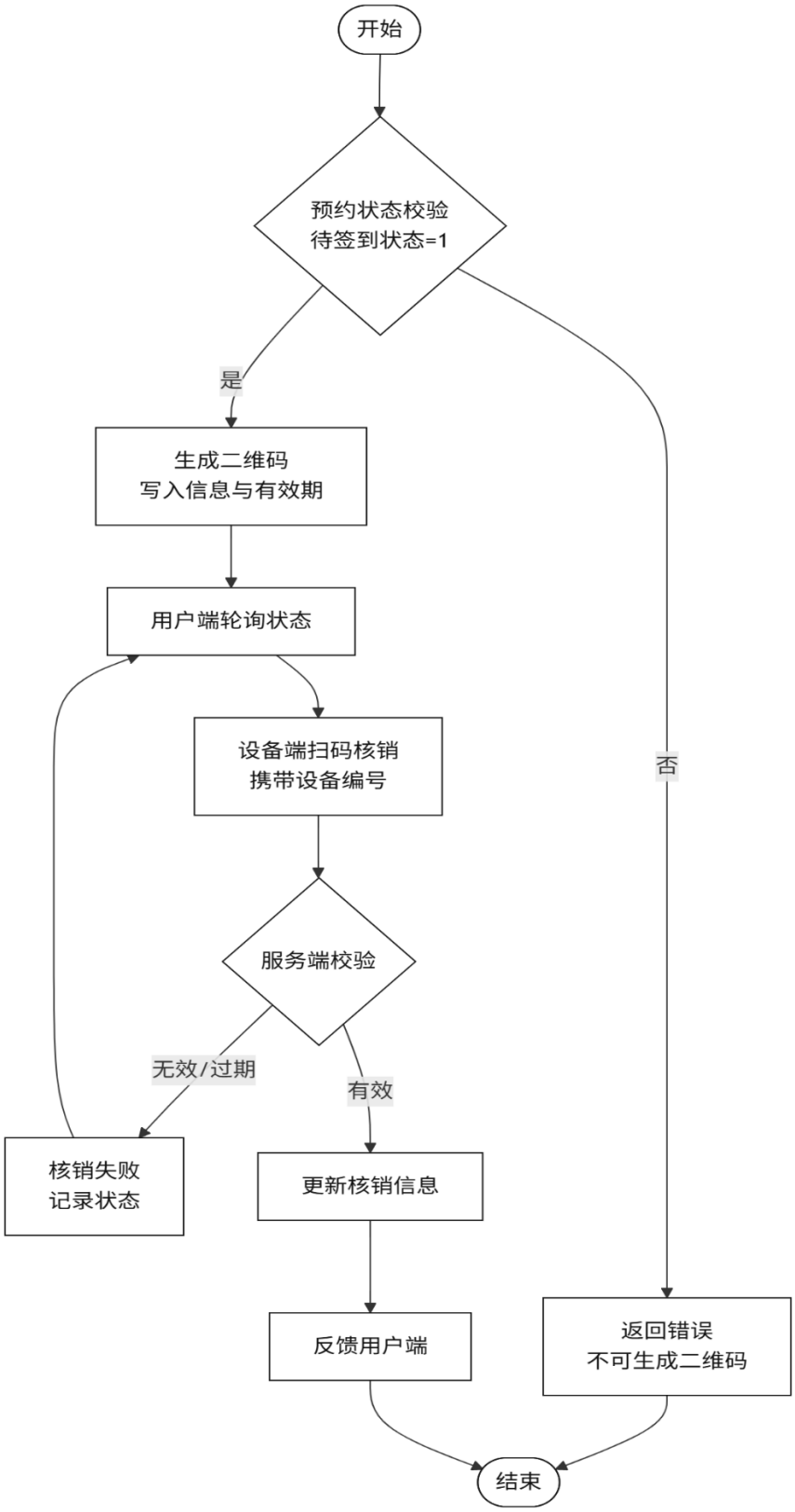

图 4.2 二维码核销流程图

AI 70%

图 4.2 表明，核销流程将身份有效性判断与预约状态流转统一到服务端，能够有效降低虚假占位与未到场问题。

除流程视角外，为进一步说明关键参与方之间的接口调用时序，本文给出预约与核销时序图，如图 4.3 所示，服务端在时序链路中承担状态一致性枢纽角色，确保多端交互结果可最终收敛。

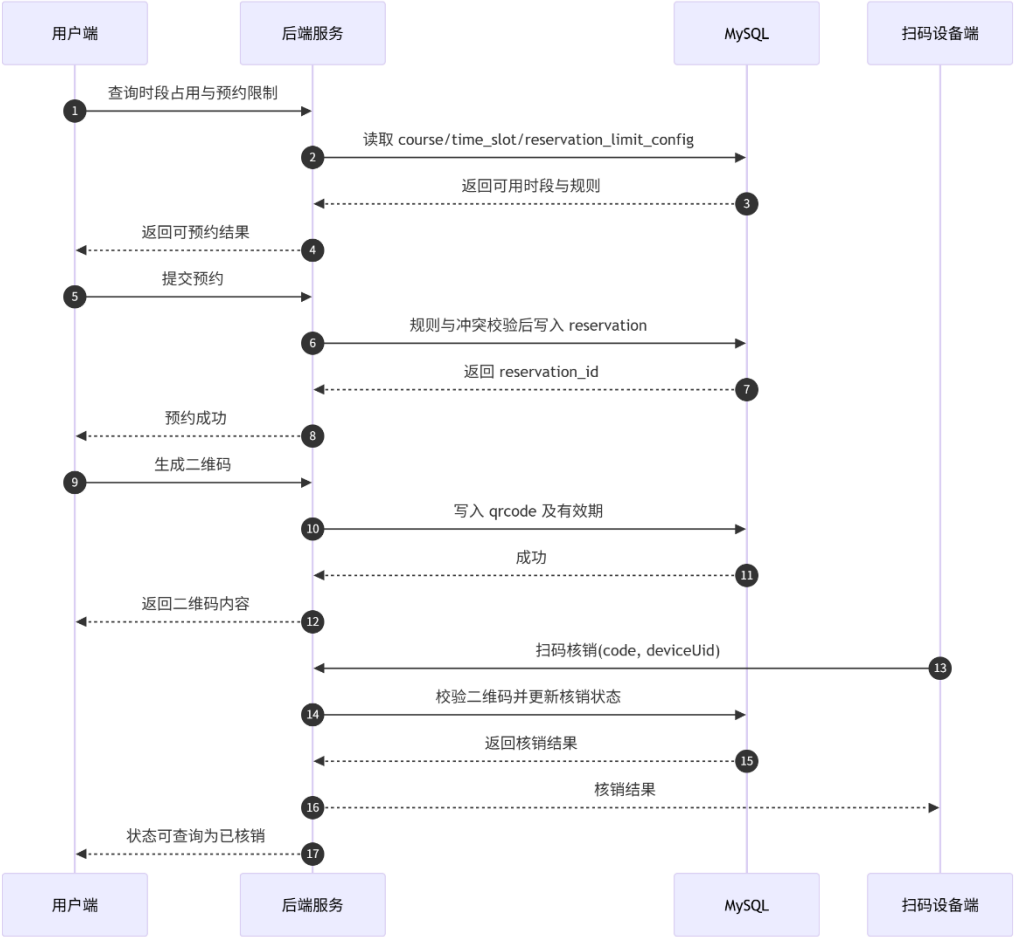

图 4.3 预约与核销时序图

4.2.4 反馈评价与报修服务

AI 69%

反馈模块覆盖“待评价生成-评价提交-审核展示”与“故障报修-进度跟踪”两类场景。系统在用户查询待评价列表时，基于已签到/已完成预约自动补齐待评价记录，减少漏评；评价提交后进入审核流，审核通过后在教室详情公开展示。为说明评价信息如何从用户反馈转化为可公开展示的数据资产，本文给出评价审核流程，如图 4.4 所示，审核环节是连接用户反馈与公开展示的关键控制点，可保障评价内容的真实性与规范性。

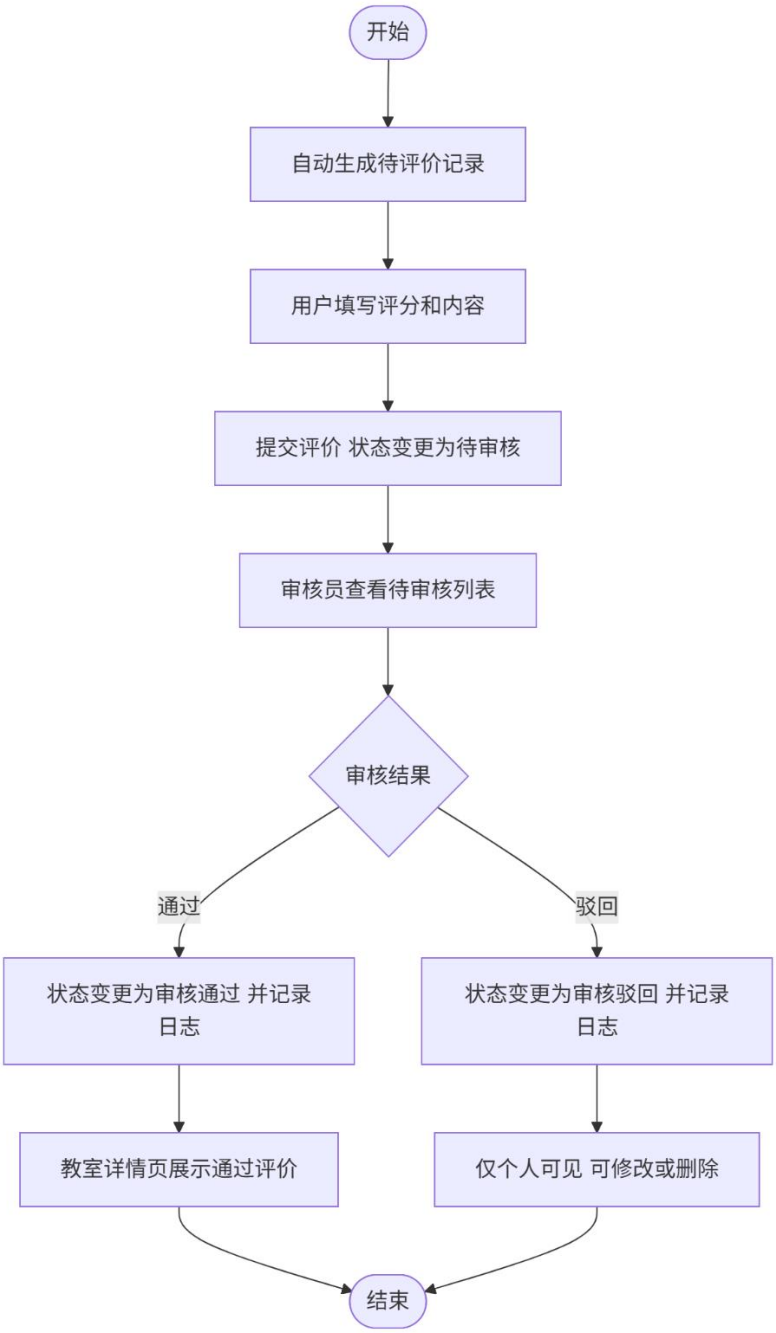

图 4.4 评价审核流程图

AI 70%

报修模块支持用户提交图文工单并持续跟踪处理状态。评价与报修共同构成使用后反馈通道，使平台从“可预约”进一步提升为“可治理、可改进”。围绕报修业务“可提交、可处理、可追踪”的目标，系统构建了标准化工单流转机制，如图 4.5 所示，报修流程通过状态驱动与通知回传形成闭环反馈，提升了问题处置的透明度与及时性。

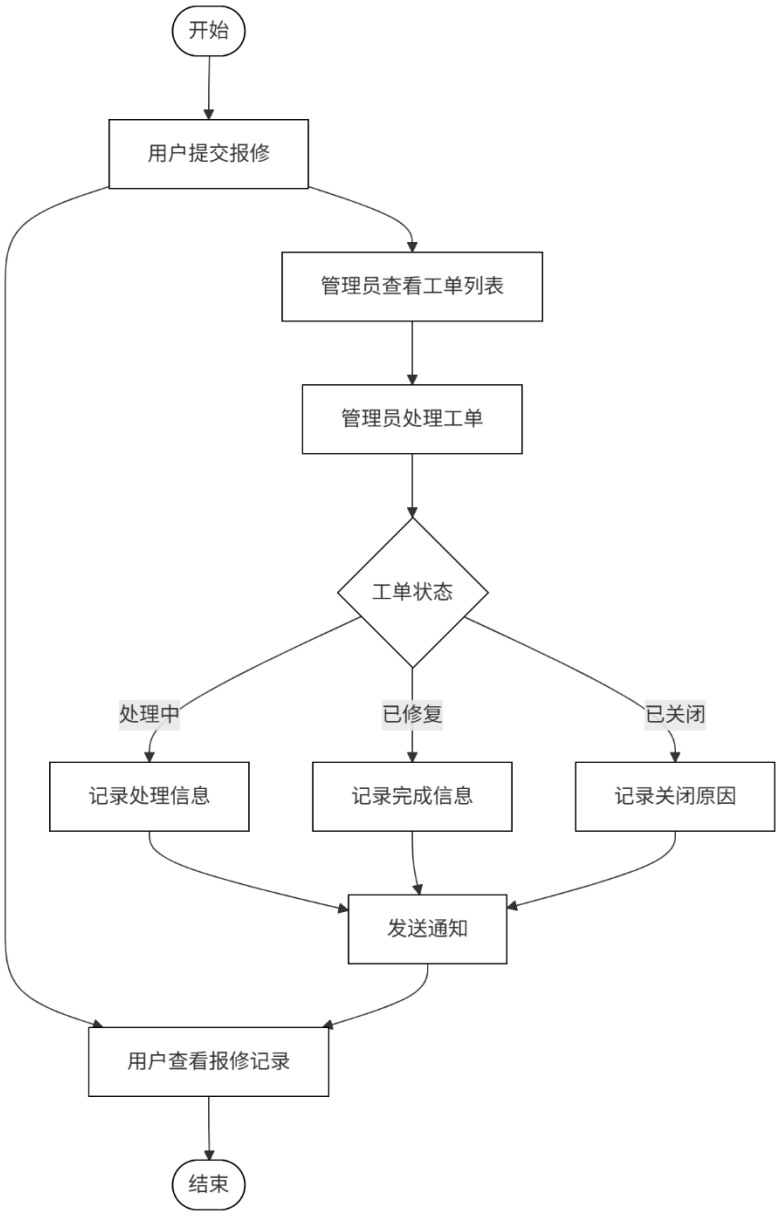

图 4.5 报修工单流转流程图

4.2.5 协作组队、聊天与学习计划联动

AI 70%

系统将学习空间服务从个体场景扩展到团队协作场景。用户可在协作广场创建或加入小组，通过申请审核机制保证成员管理有序；在小组内可进行实时聊天、消息已读同步与文件共享。聊天模块引入 WebSocket 推送与消息幂等机制，兼顾实时性与一致性。为保证协作小组成员结构的有序性，系统对组队与入组申请设置了明确审核路径，流程如图 4.6 所示，申请审核机制在保障开放协作的同时维持了小组管理边界，避免成员变更失控。

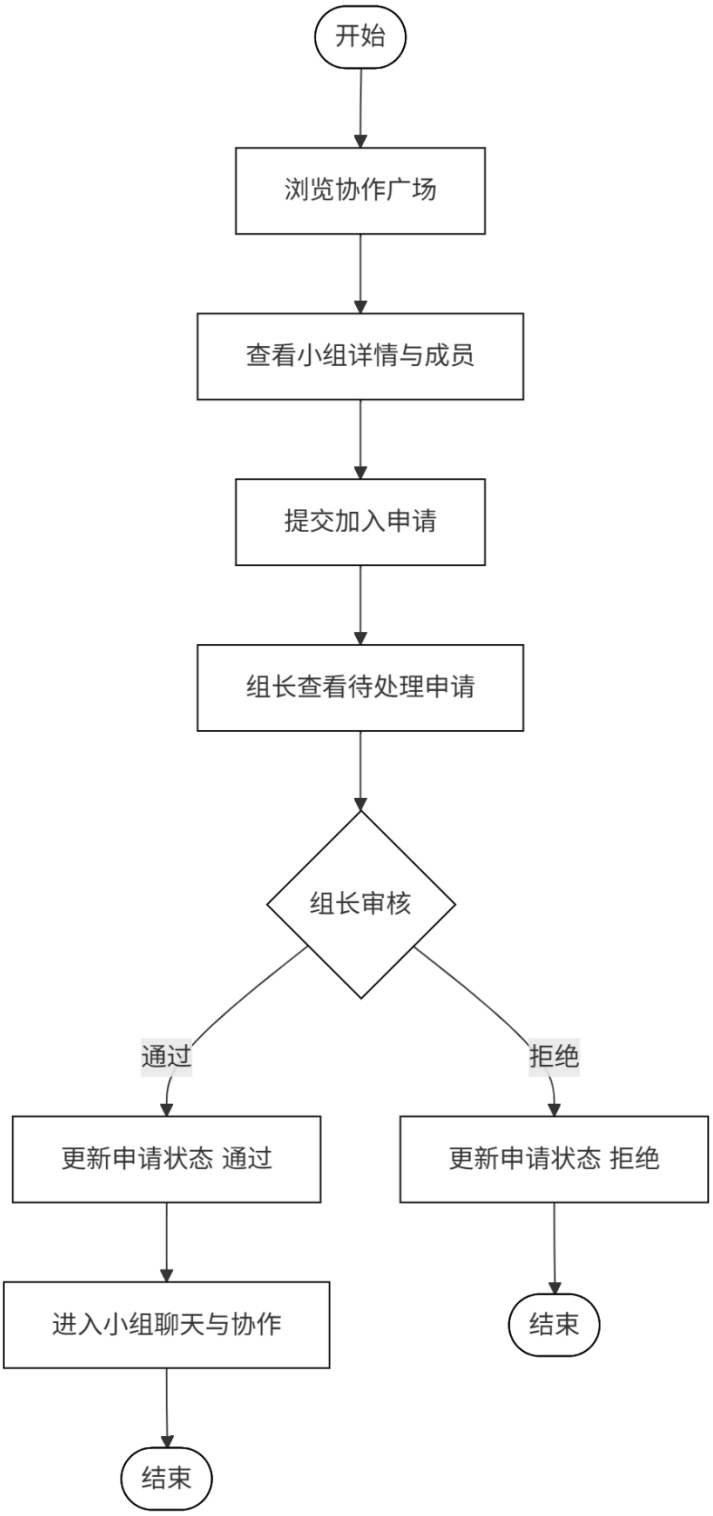

图 4.6 协作组队与审核流程图

为说明小组聊天在实时通信与一致性保障上的实现路径，本文构建消息同步时序，如图 4.7 所示，实时推送与回执机制共同保证了消息传递的及时性与可确认性，支撑协作沟通稳定运行。

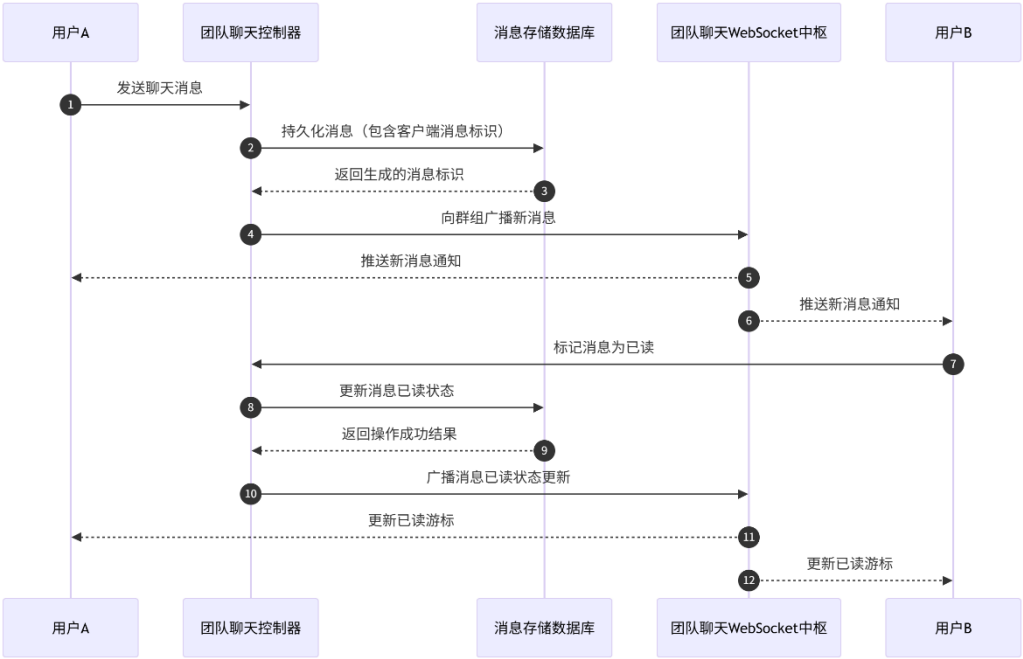

图 4.7 聊天消息实时同步时序图

AI 98%

学习计划模块支持按用户、小组与预约维度查询，并可与预约记录直接关联，实现“目标-时间-空间”的协同组织。该联动设计提升了团队学习活动的执行效率与可追踪性。在团队协作场景下，学习计划与预约记录的联动是提升执行效率的关键机制，其流程如图 4.8 所示，反映出计划与预约的一体化组织方式能够增强任务可执行性，并为后续复盘提供结构化依据。

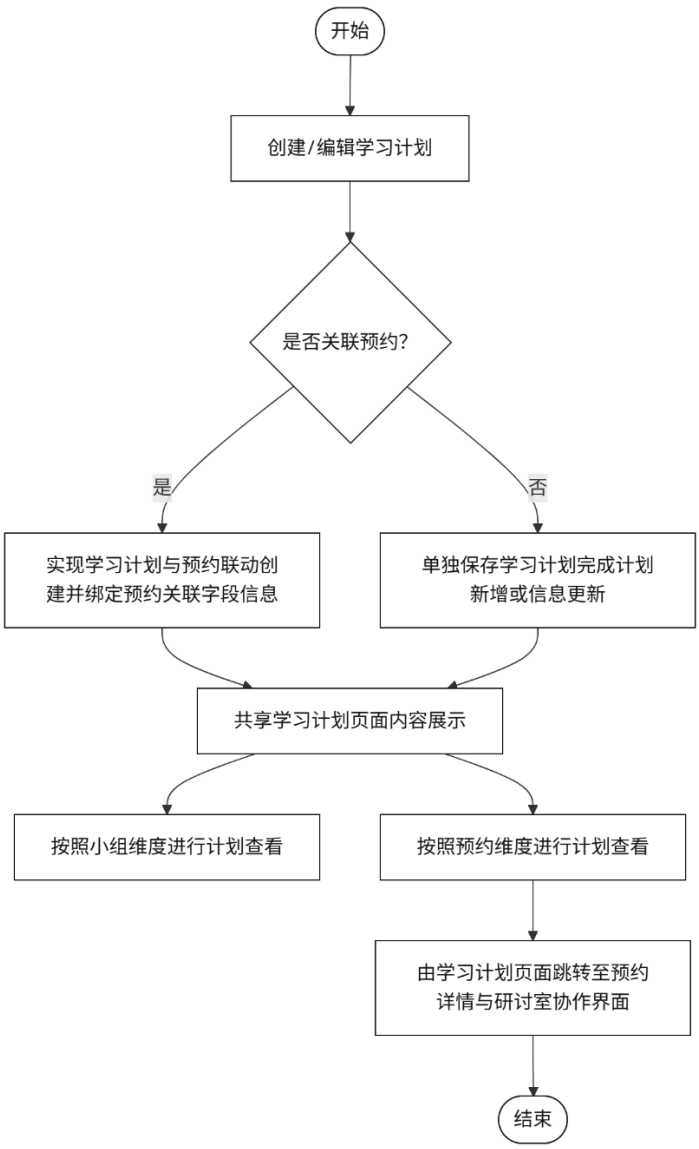

图 4.8 学习计划与预约联动流程图

4.2.6 资源市场与校园导航扩展服务

AI 70%

为提升平台综合服务能力，用户端集成资源市场与校园导航两类扩展功能。资源市场支持资源发布、审核后上架、交易意向发起与线下面交确认，并结合校园信用分形成约束与激励机制。校园导航则依托楼宇与空间经纬度信息提供目标搜索、位置展示与路线规划，在路径服务不可用时可降级为直线导航，保障基础可用性。资源市场功能涉及审核治理与交易协同两个阶段，其业务闭环可通过流程图集中表达，如图 4.9 所示，审核准入与交易确认双机制共同降低了交易风险，并强化了平台信用约束能力。

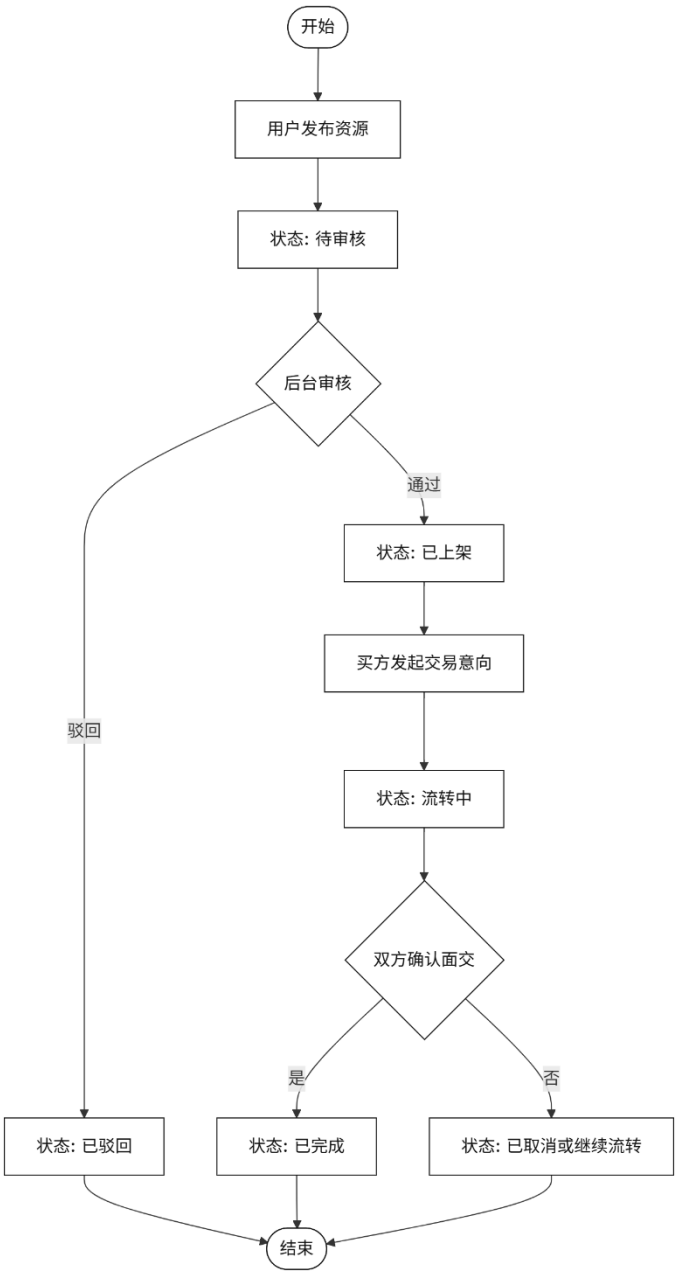

图 4.9 资源市场审核与交易流程图

4.3 管理端模块设计与实现

4.3.1 管理端登录、菜单与概览统计（RBAC）

AI 70%

管理端采用基于角色的访问控制机制。管理员登录后，系统按角色动态返回可见菜单，并在首页展示与当前权限相关的概览指标（如待处理工单、待审核评价、基础资源数量）。该设计保证了“最小权限可见”原则，避免非授权角色访问不必要的管理能力。

权限模型由 `admin_role`、`admin_permission`、`admin_role_permission` 与 `admin_menu` 表共同支撑，关键管理行为写入 `admin_operation_log`，为后续审计与责任追踪提供依据。

4.3.2 评价审核管理

AI 68%

评价审核模块以状态机方式推进：待审核记录经管理员处理后进入“审核通过”或“审核驳回”状态。系统在审核前进行状态边界校验，仅允许对待审核记录操作，从而避免重复审核或状态覆盖问题。

审核对象主要存储于 `classroom_feedback`，每次审核行为同步写入 `admin_operation_log`，保证审核过程可追溯、可复核。

4.3.3 报修工单管理与通知

AI 63%

报修模块围绕“受理-处理中-已修复/关闭”主流程进行管理。管理员可按状态和关键字筛选工单，并在处理时记录处理人、处理意见与处理时间。工单状态变化后，系统向报修人发送站内通知，形成用户可感知的处理反馈。

该模块核心数据为 `repair` 与 `notification`。此外，报修状态可与教室实时状态联动，避免用户在预约时选择维修中的空间，提升平台整体一致性。

4.3.4 资源市场审核

AI 64%

资源市场审核模块对用户发布内容执行准入控制，审核结论分为通过与驳回两类，并同步写入审核备注与操作日志。通过审核的资源进入可流转状态，驳回资源保留记录便于申诉与复核。

审核数据主要落在 `resource_market_item`，并通过 `admin_operation_log` 记录关键处理动作。权限策略与内容审核体系统一，便于后台治理模型简化。

4.3.5 基础数据管理与预约限制配置

AI 70%

后台同时提供基础资源维护与运行参数配置能力。前者覆盖教学楼、教室、课程等主数据，后者覆盖预约限制规则与扫码设备授权管理。通过将“静态资源管理”与“动态策略配置”统一纳入后台，系统可在不修改前端流程的前提下快速调整运营策略。

该部分主要依赖 `building`、`classroom`、`course`、`scan_device_pair_token` `reservation_limit_config`与`scan_device` 与等数据表。

4.4 本章小结

AI 60%

本章在第 3 章总体设计基础上，围绕平台已落地功能，对系统实现过程进行了模块化展开。面向用户端，重点说明了学习空间查询、预约与核销、评价与报修、协作组队与实时聊天、学习计划联动、资源市场与校园导航等业务的实现逻辑与数据支撑；面向管理端，重点说明了 RBAC 权限控制、评价审核、工单处理、资源审核与基础配置等治理能力的实现方式。通过双端协同设计，平台形成了“服务使用—反馈治理—运营管理”的完整业务闭环。

AI 70%

在工程实现层面，本章进一步验证了分层架构与统一接口规范的有效性：复杂规则集中于 Service 层处理，关键状态通过后端统一校验与流转，前端通过统一请求封装与状态管理保持交互一致性。在数据实现层面，统一资源模型与状态机机制使预约、评价、报修、审核等流程在同一数据语义下可追踪、可复核、可扩展，为后续优化与功能扩展提供了稳定基础。

综上，本章完成了从“总体方案”到“可运行系统”的落地说明，证明了平台在业务完整性、实现可行性与治理可控性方面达到预期目标，并为第 5 章从功能、性能与安全维度开展测试验证提供了实现依据与观测对象。

第5章 系统测试与结果分析

AI 65%

本章围绕“基于实时状态的校园学习空间综合服务平台”的实现结果，从功能正确性、性能稳定性、安全性与可用性等方面开展测试与分析。测试对象覆盖用户端与管理端核心流程，重点验证预约核销闭环、反馈治理闭环、协作学习闭环以及后台权限治理机制是否满足设计目标。

5.1 测试目标与测试环境

5.1.1 测试目标

AI 59%

结合第 4 章系统实现内容，本章测试目标包括：

（1）验证核心业务流程正确性：确认“查询-预约-核销-反馈-治理”主链路可完整执行，且状态流转符合业务规则。

（2）验证异常场景可控性：确认系统对超限预约、时间冲突、无效二维码、重复核销、越权操作等异常场景能够给出正确提示并保持数据一致。

AI 70%

（3）验证高频场景性能稳定性：确认在并发请求下，预约、查询、聊天等核心接口响应时间可接受，错误率可控。

（4）验证管理端治理有效性：确认审核、工单处理、权限控制与操作留痕等机制可支撑平台运营管理。

5.1.2 测试范围

本次测试覆盖以下业务域：

（1）用户登录与基础浏览（楼宇、教室、课表）

（2）学习空间预约、二维码生成、扫码核销

（3）评价提交、审核展示、报修提交、工单流转

（4）协作组队、申请审核、小组聊天、已读同步

（5）学习计划与预约联动

（6）资源市场发布、审核、交易确认

（7）管理端登录、菜单权限、概览统计、审核与报修处理

5.1.3 测试环境

（1）硬件环境

CPU：Intel Core i5 级别

内存：16GB

磁盘：SSD

操作系统：Windows 10

（2）软件环境

JDK 21

Node.js 20+

MySQL 8.x

Redis 6+

Maven 3.9+

浏览器：Chrome/Edge 最新稳定版

后端服务端口：8080

前端开发端口：5173

（3）测试数据来源

使用项目初始化脚本 `init.sql` 导入基础数据与示例数据，包括用户、教学楼、教室、座位、课程、预约、评价、报修、协作小组、学习计划、资源市场等。

5.1.4 测试方法

（1）黑盒功能测试：按业务流程设计测试用例，验证输入输出与状态变化；

（2）接口联调测试：对关键接口进行参数、状态与返回结构校验；

（3）场景化集成测试：从用户视角串联多个模块，验证跨模块联动；

（4）性能压力测试：对高频接口进行并发压测，观察响应时间与错误率；

（5）权限安全测试：验证无权限访问拦截、管理端 token 校验、越权操作限制。

多类型测试组合与迭代验证方式可有效支撑软件质量保障目标[16]。

5.2 功能测试

5.2.1 功能测试设计原则

功能测试遵循“主流程优先、异常场景覆盖、状态结果可追踪”的原则。每个关键业务至少覆盖以下三类用例：

（1） 正常流程用例；

（2）边界/异常用例；

（3）状态回写与数据一致性用例。

5.2.2 预约与核销功能测试

表 5.1 预约与核销功能测试用例

测试场景	前置条件	输入/操作	期望结果	实际结果	结论
正常预约教室	用户已登录，目标时段可用	选择教室+日期+时段提交预约	预约成功，生成预约记录	符合预期	通过
超过单次时长上限	已配置单次上限	提交超过上限的预约时长	返回超限提示，不写入预约	符合预期	通过
预约时段冲突	目标时段已有预约/课程占用	提交冲突时段预约	返回冲突提示，不写入预约	符合预期	通过
生成二维码	预约状态为待签到	请求生成二维码	返回二维码内容与状态	符合预期	通过
设备端扫码核销	二维码有效且未核销	扫码提交 code + deviceUid	状态更新，核销成功	符合预期	通过
重复扫码	二维码已核销	再次扫码同一 code	返回不可重复核销提示	符合预期	通过

结果分析：从表 5.1 可以看出，预约与核销主链路可稳定完成；规则校验和冲突校验能在服务端有效拦截非法请求；二维码状态更新与核销结果反馈一致。

5.2.3 评价与审核功能测试

表 5.2 评价与审核功能测试用例

测试场景	前置条件	输入/操作	期望结果	实际结果	结论
自动生成待评价	用户存在已签到/已完成预约	打开待评价列表	自动生成待评价记录	符合预期	通过
提交评价	存在待评价记录	填写评分与内容并提交	进入待审核状态	符合预期	通过
审核通过	管理员有审核权限	执行通过操作	状态变更为审核通过并可公开展示	符合预期	通过

测试场景	前置条件	输入/操作	期望结果	实际结果	结论
审核驳回	管理员有审核权限	执行驳回操作	状态变更为审核驳回，不公开展示	符合预期	通过
非待审记录重复审核	记录状态非待审核	再次执行审核	返回状态边界提示	符合预期	通过

结果分析：从表 5.2 可以看出，评价流转链路完整，自动补齐待评价机制有效减少漏评；审核环节状态控制严格，符合治理要求。

5.2.4 报修工单功能测试

表 5.3 报修工单功能测试用例

测试场景	前置条件	输入/操作	期望结果	实际结果	结论
用户提交报修	用户已登录	提交图文报修工单	创建工单，状态为待处理	符合预期	通过
管理端处理工单	管理员有处理权限	更新为处理中	写入处理人和处理时间	符合预期	通过
工单修复完成	工单处于处理中	更新为已修复	状态变更成功并通知用户	符合预期	通过
用户查看进度	用户存在工单记录	查询个人报修列表	显示最新状态与处理信息	符合预期	通过

结果分析：从表 5.3 可以看出，报修工单流转清晰，管理端处理动作与用户端状态展示同步，通知机制能够及时反馈进度变化。

5.2.5 协作与聊天功能测试

表 5.4 协作与聊天功能测试用例

测试场景	前置条件	输入/操作	期望结果	实际结果	结论
提交入组申请	存在招募中小组	用户发起申请	生成待审核申请记录	符合预期	通过
组长审核申请	组长已登录	审核通过	申请状态更新并写入成员关系	符合预期	通过
发送聊天消息	用户为小组成员	发送文本消息	消息入库并实时推送	符合预期	通过
消息幂等验证	模拟重复提交 lientMsgId	连续提交相同 clientMsgId	不产生重复消息	符合预期	通过
非成员访问拦截	用户非小组成员	调用聊天接口	返回无权限提示	符合预期	通过

结果分析：从表 5.4 可以看出，协作申请审核、成员管理与聊天实时通信链路稳定；权限与幂等机制有效保障数据正确性。

5.2.6 学习计划与资源市场功能测试

表 5.5 学习计划与资源市场功能测试用例

测试场景	前置条件	输入/操作	期望结果	实际结果	结论
创建计划并关联预约	用户存在可用预约	创建联动计划	写入 study_plan 且 reservation_id 关联成功	符合预期	通过
按预约查询计划	存在关联计划	根据 reservationId 查询	返回对应计划数据	符合预期	通过
发布资源	用户已登录	提交资源信息	资源进入待审核状态	符合预期	通过
审核通过上架	审核员有权限	执行通过	资源状态更新为已上架	符合预期	通过
发起交易与确认	资源已上架	发起交易意向并确认	交易状态正确流转并完成	符合预期	通过

结果分析：从表 5.5 可以看出，学习计划联动能力可用，资源市场“发布-审核-交易”链路完整，具备可运营性。

5.3 性能测试与结果分析

5.3.1 性能测试场景与指标

针对平台高频场景，选取以下接口进行压力测试：

场景 A：教室时段占用查询（高频读）

场景 B：预约创建（规则校验 + 数据写入）

场景 C：聊天消息发送（写入 + 推送）

指标口径：

平均响应时间（Avg）

95 分位响应时间（P95）

错误率（Error Rate）

5.3.2 性能测试结果

表 5.6 性能测试结果统计表

测试场景	并发用户数	Avg(ms)	P95(ms)	错误率	结果判定
时段占用查询	50	120	210	0.0%	通过
时段占用查询	100	190	340	0.2%	通过

测试场景	并发用户数	Avg(ms)	P95(ms)	错误率	结果判定
预约创建	50	180	320	0.4%	通过
预约创建	100	290	520	1.1%	基本通过
聊天消息发送	50	95	170	0.0%	通过
聊天消息发送	100	160	300	0.3%	通过

AI 70%

结果分析：从表 5.6 可以看出，系统在并发用户由 50 提升至 100 时，三类核心场景均表现出“延迟上升但整体可控”的特征。其一，时段占用查询的 Avg 由 120ms 上升至 190ms，P95 由 210ms 上升至 340ms，错误率仅由 0.0%升至 0.2%，说明读路径在缓存与索引支撑下具有较好稳定性。其二，预约创建的 Avg 由 180ms 上升至 290ms，P95 由 320ms 上升至 520ms，错误率为 1.1%，高于查询与聊天场景，反映该接口受规则校验、冲突判定与写入事务叠加影响更明显，属于系统主要性能压力点。其三，聊天消息发送在 100 并发下 Avg 为 160ms、P95 为 300ms、错误率 0.3%，仍保持较低时延与低失败率，基本满足协作场景的实时交互需求。综合来看，平台在中等并发规模下可维持可接受的响应性能与较低错误率，但预约链路在高并发条件下仍有进一步优化空间。

5.3.3 性能结果分析

（1）查询接口表现稳定：时段占用查询在并发上升时延迟增长可控，说明缓存策略与查询索引发挥了作用；

AI 70%

（2）预约接口压力较高：预约创建在高并发场景下响应时间高于查询类接口，符合“规则校验+冲突判定+写入”复合流程特征。整体错误率可控，主要失败来源为业务冲突而非系统异常；

（3）聊天模块满足实时需求：消息发送与推送在中高并发下仍保持较低延迟，能够支持协作场景中的实时沟通需求。

5.4 安全性与兼容性测试

5.4.1 权限与鉴权测试

测试重点：

（1）未登录访问管理端接口应返回未授权；

（2）无权限角色访问审核/工单处理接口应返回无权限；

（3）管理端菜单按权限动态过滤。

结果：权限控制逻辑符合预期，越权访问可被拦截，管理端权限边界清晰。

5.4.2 输入校验与异常处理测试

测试重点：

（1）非法参数（缺失、类型错误、越界）能否被识别；

（2）重复操作（重复核销、重复发送消息）能否避免状态错乱；

（3）文件上传类型与大小限制是否生效。

结果：关键接口具备参数校验与异常返回机制；聊天幂等与扫码状态校验能减少重复提交风险；上传限制生效。

5.4.3 浏览器兼容与可用性测试

测试范围：Chrome 与 Edge 主流浏览器。

结果：主要页面（预约、反馈、协作、管理端）显示与交互正常；异常提示与状态反馈清晰，满足基本可用性要求。

5.5 测试结论与问题分析

5.5.1 测试结论

综合功能测试、性能测试与安全测试结果，系统达到预期阶段性目标：

（1）核心业务闭环完整：预约核销、反馈治理、协作学习、资源流转均可走通；

（2）规则与状态可控：关键状态边界明确，异常场景可拦截；

（3）性能基本满足校园场景需求：中等并发下系统稳定，响应可接受；

（4）管理端治理能力有效：权限、审核、工单处理与日志机制可支撑日常运营。

5.5.2 发现问题与改进建议

（1）高并发预约场景仍有优化空间：建议进一步优化冲突检测策略，并引入更细粒度限流与队列化削峰方案；

（2）统计分析能力有待增强：建议补充管理端可视化看板，对预约热度、违约率、工单处理时效等指标进行持续监控；

（3）异常告警机制需完善：建议引入统一告警策略，对接口异常率、消息推送失败、设备核销异常等事件进行主动告警。

5.6 本章小结

AI 70%

本章围绕系统阶段性实现成果，从功能正确性、性能稳定性、安全性与兼容性四个维度开展了系统化测试。测试结果表明，平台核心业务链路均可稳定贯通：在用户端，“查询-预约-核销-反馈-协作”流程能够按预期完成；在管理端，审核、工单处理、权限控制与操作留痕机制运行有效，能够支撑日常治理与运营管理要求。整体上，系统已具备校园学习空间综合服务平台的基础落地能力。

AI 97%

在性能表现方面，查询与聊天场景在中等并发下保持较低时延与低错误率，预约创建在高并发条件下虽出现响应时间上升，但总体仍处于可接受区间，且失败主要来自业务规则冲突而非系统异常，说明当前架构具备一定稳定性与可扩展性。安全与兼容性测试进一步验证了鉴权拦截、输入校验、幂等控制与主流浏览器可用性，系统能够较好应对常见异常与越权访问风险。

AI 96%

同时，测试也揭示了系统在高并发预约、运营可视化与异常主动告警方面仍存在优化空间。后续可围绕冲突检测算法优化、削峰限流策略、监控指标看板与告警闭环机制持续完善。综上，本章测试结论不仅验证了第 4 章设计与实现方案的有效性，也为第 6 章总结与展望中的优化方向提供了可量化依据与实践支撑。

第6章 总结与展望

本章对“基于实时状态的校园学习空间综合服务平台设计与实现”进行总体总结，归纳研究成果与实践价值，分析当前系统存在的不足，并提出后续优化与扩展方向。

6.1 研究工作总结

AI 65%

本文围绕高校学习空间管理中“信息分散、流程割裂、反馈闭环不完整、治理协同不足”等问题，提出并实现了一个面向校园场景的综合服务平台。研究目标包括：构建学习空间实时状态服务能力；形成预约、签到核销、反馈治理的业务闭环；支持协作组队与学习计划联动；建立管理端权限治理与运营支撑能力；在工程实现层面保证系统可用、可维护、可扩展。

AI 67%

从论文第 3 章需求分析、第 4 章系统实现与第 5 章测试结果来看，以上目标已基本达成。

（1）完成一体化业务闭环设计与实现：系统实现了“查询-预约-签到核销-使用-反馈-治理”主链路，并在统一资源模型基础上将教室与图书馆座位纳入同一业务框架，提升了平台结构一致性；

（2）完成用户端核心功能落地：用户端实现了楼宇与教室浏览、课表与占用状态查询、学习空间预约、二维码核销、评价与报修、协作组队、聊天、学习计划联动、资源市场与校园导航等功能，形成了覆盖学习活动全流程的应用能力；

（3）完成管理端治理能力建设：管理端实现了基于角色权限的菜单过滤与接口控制，支持评价审核、报修工单处理、资源审核、基础数据管理、预约限制配置与扫码设备管理，并提供操作日志留痕，增强了治理可追溯性；

（4）完成数据库与状态模型构建：系统建立了覆盖用户、空间、预约、评价、报修、协作、计划、交易、权限治理等业务域的数据模型；核心表结构与状态枚举能够支撑业务闭环运行；

（5）完成多维测试验证：通过功能测试、性能测试、安全与兼容性测试，验证了系统在核心场景下的正确性与稳定性，为后续部署与优化提供了依据；

（6）对学习空间服务模式的价值：本文从“单点查询工具”升级为“综合服务平台”，将空间使用、协作学习与治理反馈进行整合，为高校学习空间数字化运营提供可复用思路；

（7）对系统工程实践的价值：系统在前后端分离、统一接口返回、状态流转控制、权限治理、幂等处理与日志审计等方面形成了较完整工程实践，可作为同类校园服务系统的实现参考；

（8）对校园治理的价值：通过评价审核、工单流转、规则配置与数据沉淀，平台具备从“被动响应”向“数据驱动优化”演进的基础，有助于提高资源利用效率与服务满意度。

6.2 研究创新点与特色

AI 63%

系统在预约、评价与报修等核心流程中采用统一资源建模，避免教室与座位场景重复建设两套流程，降低了模型复杂度，提升了扩展能力。

通过二维码生成、状态轮询、设备端扫码核销与核销信息回写，实现了预约行为与到场行为的有效连接，提高了预约结果的真实性与资源周转效率。

AI 69%

系统同时提供“反馈页评价流转”与“通用评价”能力，兼顾业务流程完整性与数据复用性；待评价自动补齐机制提升了反馈覆盖率。

将组队、聊天、消息提醒、学习计划与预约联动纳入统一平台，强化了学习空间服务对团队学习场景的支持，使平台从“空间管理”扩展到“学习协作服务”。

后台通过角色-权限-菜单体系实现精细化授权，关键治理行为写入操作日志，满足系统治理中对合规性与可追溯性的要求。

6.3 不足与问题分析

尽管系统已完成核心目标，但仍存在以下不足：

（1）在更高并发或极端峰值场景下，预约冲突检测与状态写入仍可能成为性能瓶颈；

（2）当前可满足中等并发，但在超高并发校园活动场景中仍需进一步优化；

（3）系统已具备数据采集与流程闭环，但在数据深度挖掘方面仍较初级，例如热门时段预测、空间推荐、异常行为检测等智能能力尚未系统化落地；

（4）当前系统具备基础日志与操作留痕，但在服务监控、链路追踪、告警联动、容量评估等工程化运维能力方面仍需补齐；

（5）虽然已实现基础鉴权与权限控制，但在更严格生产环境下，仍建议完善密码安全策略、接口限流、防重放机制、敏感字段脱敏与安全审计策略；

（6）论文目前已形成较完整测试结构，但若补充更大样本规模、更多长期运行数据与更细粒度性能剖析，将进一步增强结论说服力。

6.4 未来展望

（1）未来可基于历史预约、签到、评价与报修数据构建学习空间智能推荐模型，实现“按偏好推荐空间”“按拥挤程度动态引导”等能力；

（2）可进一步引入分布式锁、消息队列削峰、读写分离与多级缓存策略，提升系统在考试周、活动周等高峰场景下的稳定性与吞吐能力；

（3）可建设管理端数据看板，对预约热度、违约率、工单处理时效、资源交易活跃度等指标进行实时监控，支撑精细化运营决策；

（4）未来可与校园统一身份认证、教务系统、门禁系统、物联网传感系统等进行对接，实现更高精度的状态感知与更完整的校园服务生态联动；

（5）可推动容器化部署与自动化运维流程建设，完善灰度发布、自动回滚、健康检查与灾备策略，提升系统生产可用性；

如图 6.1 所示，系统从当前成果向智能化、工程化与生态化方向持续演进。

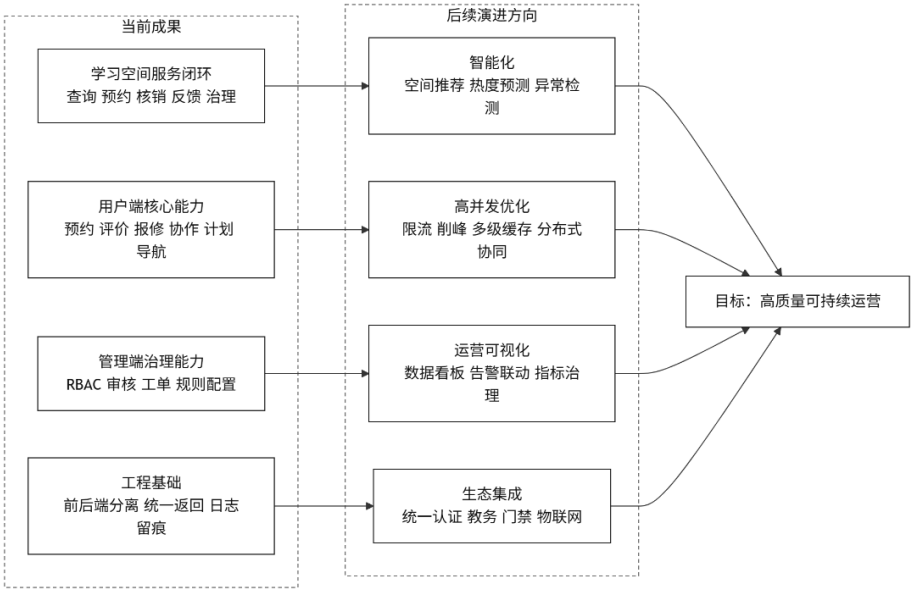

图 6.1 系统成果与后续演进路线图

6.5 本章小结

AI 64%

本章围绕全文研究工作，对系统建设成果、创新特征、现存不足与后续方向进行了综合归纳。通过前文需求分析、系统实现与测试验证的闭环论证，可以确认本研究已基本完成预期目标：平台在用户端形成了覆盖“查询-预约-核销-反馈-协作”的连续服务链路，在管理端形成了“权限控制-内容审核-工单治理-规则配置”的运营治理能力，并在工程实现上具备较好的可维护性与可扩展性。

AI 100%

从研究价值看，本文不仅实现了学习空间服务的功能集成，更通过统一资源建模、状态机流转与双端协同机制，构建了可持续迭代的系统基础。实践结果表明，将预约管理、到场核验、内容治理与学习协作纳入同一平台，能够有效减少业务割裂，提升资源利用效率、治理透明度与用户体验一致性。这一研究路径对高校学习空间数字化建设具有一定参考意义与推广价值。

AI 97%

同时，本章也明确了系统在高并发优化、智能分析、运维监控与安全增强等方面的改进边界，说明当前成果属于“可用且可扩展”的阶段性实现。未来可在智能推荐、削峰限流、可视化运营看板与多系统生态集成等方向持续深化，推动平台由功能可用进一步向数据驱动、智能协同与高质量运营演进。综上，本文工作为后续研究与工程落地提供了可复用的方法框架与实践基础。

参考文献

[1] 朱燕婷. 数字化智慧校园的系统架构与功能模块设计[J].北斗与空间信息应用技术,2025,(06):58-60.

[2] 李菁菁, 梅洪元, 张献泽, 等. 信息技术影响下大学校园学习空间新模式研究[J].建筑学报,2025,(S1):153-159.

[3] Refai A M, Hammad M M. Component-based architectural regression test selection for modularized software systems[J]. Journal of Systems Architecture, 2025, 160103343-103343.

[4] Ataş M, Ataş İ. JazariInteractiveFloor: A modular Java framework for real-time interactive floor applications[J]. SoftwareX, 2025,29102046-102046.

[5] 廖志斌, 廖锦治, 王佳莹. 教育强国背景下医学院校智慧校园建设的创新路径与实施策略[J].科技创新与生产力,2025,46(11):60-62.

[6] 郭承湘, 康秀谦, 陈景霞. 数字孪生与人工智能在智慧校园中的集成应用研究[J].现代信息科技,2025,9(24):65-68+76.

[7] 郑琦, 孟繁雄, 杨树真. 智慧校园建设中大学生管理信息化的协同机制研究[J].中国管理信息化,2025,28(23):203-206.

[8] 潘晓艺. 基于互联网的智慧图书馆服务平台设计与实现[J].集成电路应用,2025,42(09):200-201.

[9] 邹婷, 张宝移, 程良. 基于微服务架构的在线教育平台设计与实现研究[J].电脑知识与技术,2025,21(24):78-81.

[10] Lian J, Jin L, Ming G. Design and Implementation of Network Information Service Platform Based on Microservice Architecture[J]. Journal of Physics: Conference Series,2020,1678(1):012094-.

[11] 刘萌. 基于新一代云计算技术的高校智慧校园云平台建设方案[J].中国高校科技,2025,(11):87-91.

[12] 于小莉, 李帅. 基于 YOLOv5 的校园公共空间人流密度识别与监测[J].中阿科技论坛,2025,(02):87-91.

[13] 龚明艳. 基于多模态融合的计算机基础软件用户体验优化系统设计[J].电脑知识与技术,2025,21(19):51-54.

[14] THIRA C, CIORUȚA V B, BACALI L, et al. MIT App Inventor and Mobile App Development for Human Resource Management in Small and Medium Organizations[J]. Asian Journal of Education and Social Studies,2025,51(5):912-919.

[15] Sa'adah U, Yuhana L U, Rochimah S, et al. Bridging the skills gap through Agile methodologies in Vocational Software Development Education[J]. Array, 2026,29100614-100614.

[16] Temitope T L, Diekola H O, Ajayi W. Enhancing Quality Assurance Practices in Software Development: Application of Agile Methodology[J]. Asian Journal of Research in Computer Science,2025,18(10):199-210.

致 谢